



Protección del jugador: la caja negra de la lógica

Por

Alberto Peñalba Martos - Nota: 10

Dorje Khampa Herrezuelo Blasco - Nota: 10



UNIVERSIDAD COMPLUTENSE
MADRID

Dirigido por

José Luis Vázquez Poletti
David Pacios Izquierdo

MADRID, 2025–2026

AGRADECIMIENTOS

A nuestros directores, por su guía paciente, su criterio y la generosidad con la que nos han acompañado a lo largo de este trabajo. Sin su tiempo, su exigencia y su confianza, estas páginas no serían lo que hoy son.

DE ALBERTO

A todas las personas que, de una forma u otra, han contribuido al desarrollo y realización de este trabajo.

También valoro especialmente la experiencia y el aprendizaje adquiridos durante la elaboración de este proyecto

— Alberto Peñalba Martos —

DE DORJE

A mi madre, por su esfuerzo callado y su amor incondicional, por sostenerme en cada paso y creer en mí cuando más falta hacía.

A Lulú y a Poppy, compañeras fieles de tantas horas, que con su sola presencia hacían el mundo más amable.

A mi tío José Luis y a mi tía Toñi, por su cariño y su presencia, por estar ahí, siempre, sin necesidad de palabras.

Y a mi tía Julia, que en paz descanse, por todo lo que me dio y que aún llevo conmigo; porque hay personas que no se van del todo nunca.

— Dorje Khampa Herrezuelo Blasco —

Resumen

La industria del videojuego vive una paradoja. Por un lado, se ha convertido en el motor más poderoso de la economía del entretenimiento digital, superando al cine y la música juntos [1]. Millones de jugadores se conectan a diario para competir, cooperar y construir comunidades globales. Sin embargo, esta misma industria se enfrenta a una amenaza persistente: la inseguridad.

Trampas, fraudes en puntuaciones, cuentas robadas, economías virtuales alteradas y copias ilegales forman parte de un escenario habitual. El problema es estructural: durante años, muchos proyectos —especialmente en el ámbito independiente y académico— han delegado su seguridad en el cliente, asumiendo erróneamente que el dispositivo del jugador es confiable. Esta práctica ha dado lugar a vulnerabilidades que comprometen la integridad de los datos y la experiencia de juego.

Este Trabajo Fin de Grado propone un enfoque alternativo: diseñar videojuegos con seguridad incorporada desde el inicio, aplicando principios de computación distribuida y aprovechando arquitecturas *serverless* en la nube. El objetivo es trasladar la autoridad de validación al servidor y garantizar la coherencia, trazabilidad y autenticidad de los datos del jugador mediante mecanismos reproducibles y escalables. Con ello se busca ofrecer un marco que permita a los desarrolladores construir experiencias más seguras y sostenibles sin comprometer la agilidad del desarrollo.

Palabras clave

Ciberseguridad, videojuegos, arquitectura *serverless*, computación distribuida, AWS, DRM, anti-cheat, validación, backend seguro, nube.

Abstract

The video game industry lives in a paradox. On one hand, it has become the most powerful engine of the digital entertainment economy, surpassing film and music combined [1]. Millions of players connect every day to compete, cooperate, and build global communities. Yet this same industry faces a persistent threat: insecurity.

Cheating, score fraud, stolen accounts, altered virtual economies, and illegal copies are now part of the gaming landscape. The root cause is structural: for years, many projects—especially in the independent and academic sectors—have relied on the client for validation, assuming that the player’s device is trustworthy. This approach has led to vulnerabilities that compromise data integrity and player trust.

This Final Degree Project proposes an alternative approach: building games with security integrated from the start by applying distributed computing principles and leveraging *serverless* cloud architectures. The goal is to move validation authority to the server and ensure data coherence, traceability, and authenticity through reproducible and scalable mechanisms. The result is a framework that enables developers to create secure and sustainable gaming experiences without sacrificing agility.

Keywords

Cybersecurity, video games, *serverless* architecture, distributed computing, AWS, DRM, anti-cheat, validation, secure backend, cloud computing.

Índice general

Capítulo 1	Introduction	1
Capítulo 1	Introducción	7
Capítulo 2	Estado de la Cuestión	13
Capítulo 3	Tecnologías utilizadas y Metodología	19
Capítulo 4	Arquitectura y Diseño	27
Capítulo 5	Casos de Uso y Observación	39
Capítulo 6	Conclusiones y trabajo a futuro	49
Capítulo 6	Conclusions and future applications of the project	53
Anexos		57
Bibliografía		66

Chapter 1. Introduction

Motivation

The video game industry constitutes one of the driving axes of the contemporary digital economy. Historically, it has surpassed the film and music industries combined in terms of revenue volume and global penetration [1]. What emerged as a domestic *hobby* in the early 1970s has become a cultural, ecosocial, and technological phenomenon, supported by international communities, high-performance competitions (*eSports*), and complex virtual economies.

Fraudulent score alterations, the injection of *cheats*, manipulation of the client state, and the distribution of unauthorised copies represent everyday threat vectors. Legitimate users trust in the fairness of the interactive experience and in their progression being assessed rigorously. Nevertheless, much of the infrastructure that supports this trust is trivially exploitable due to underlying architectural vulnerabilities.

This situation not only damages the way games are played, but also harms the reputation of developers and player communities.

Historically, various countermeasures have been deployed to mitigate these vulnerabilities. One of the first lines of defence consisted of integrating *Digital Rights Management (DRM)* mechanisms. These mechanisms were conceived to restrict unauthorised copying and execution of *software*. The evolution from simple activation-key checks to complex dynamic obfuscators such as *Denuvo Anti-Tamper* evidences an ongoing arms race between the protection of intellectual property and reverse engineering.

In parallel, the development of intrusion-detection systems specific to video games (*anti-cheat*) has proliferated. These solutions have undergone notable sophistication. Whereas in their initial iterations they were limited to static verification in user space, they now predominantly operate at the *kernel* level of the operating system (for example, *BattlEye* or *Easy Anti-Cheat*), theoretically increasing friction against memory manipulation.

Even so, these mechanisms share a structural limitation: they rely heavily on the integrity of the client. No matter how sophisticated they are, as long as the control point resides on the player's device, an attack surface will always remain open. The motivation for this work arises precisely from that weakness: rethinking where the authority of the system should reside, instead of relying on the rationality of the player.

This structural weakness manifests itself recurrently in practice. A documented recent case is that of February 2026, when MKDEV released a *crack* for *Resident Evil Requiem*, protected by *Denuvo Anti-Tamper*. In order to bypass the protection, the *crack* required disabling security features integrated by default in *Windows* environments, such as *Secure Boot*, *VBS*, *HVCI*, *Hyper-Visor*, and the memory integrity feature of *Windows Defender*, as well as enabling the loading of *drivers* in *Test-Mode*. This configuration leaves the machine exposed to *kernel-level rootkits* that are not detectable by either the user or the operating system; in this specific case, the distributed *crack* also included a *keylogger*. Beyond the

anecdote itself, the episode illustrates a more general pattern: when the control point resides on the client, the costs that the user is willing to assume in order to circumvent the protection may end up compromising the security of the operating system itself.

It is therefore proposed that intrinsically more secure game topologies can be designed by adopting principles of distributed computing and *serverless*¹ architectures, shifting logical validation authority and critical state decisions from the client to the *backend*. This paradigm shift not only safeguards session integrity, but also establishes an auditable, deterministic, and sustainable framework.

Problem and gap

The underlying problem does not lie in a lack of cryptographic or technological primitives, but in the prevailing architectural paradigm. In many developments — particularly in *indie* or academic environments — there is an overdependence on client integrity: scores, movement vectors, or inventories originating on the user's machine are validated *de facto* without subsequent authoritative reconciliation on the server. Although this *client-authoritative* model accelerates the initial development cycle, it introduces a massive attack vector that compromises the long-term viability of the ecosystem.

The result of this trend is a fragmented and uneven ecosystem. Games end up being only as secure as the weakest user device. Data tampering, *exploits*, and cheating cease to be isolated incidents and become symptoms of a poorly conceived architecture. Furthermore, the dependence on local solutions makes it impossible to scale or maintain systems coherently, increasing technical debt and operational risk.

A twofold gap can therefore be identified:

- **Technical gap:** generated by the lack of reference architectures (or *blueprints*) that democratise the deployment of *server-authoritative* solutions in teams with limited resources.
- **Methodological gap:** due to treating security as a final stage of development rather than as a design principle (*security by design*).

This work seeks to contribute to closing that gap by defining an architectural approach that prioritises server-side verification, observability, and data consistency, without sacrificing agility or the quality of the experience, always within a *serverless* environment.

Scope and context of the work

This Final Degree Project does not aim to solve every security problem in the industry, but rather to address a specific and representative area: the protection of the critical elements of a video game through a secure and scalable *backend*².

The scope is delimited around the following axes:

¹A cloud execution model in which infrastructure provisioning and scaling are managed dynamically by the provider, abstracting the physical layer.

²The part of an application or website that remains invisible to the end user.

- It focuses on the design and evaluation of an architecture that manages licences, progression, scores, and authentication from the server.
- A *kernel-level anti-cheat* system is not developed, although its relevance is acknowledged as part of the historical context.
- New *DRM* algorithms are not designed, but they are analysed as precedents in the control of client integrity.
- Real-time multiplayer networks and latency-related aspects are not addressed, since the focus lies on logical security and the consistency of the game state.

From a contextual standpoint, this work falls within two complementary frameworks:

- **Academic and educational:** as it is a project oriented towards applied learning in cybersecurity, systems architecture, and video game development.
- **Practical and replicable:** as it offers a viable model for independent studios or small-scale projects, where security is often relegated due to time or resource constraints.

The value of this research is considered to lie precisely in this dual perspective: combining academic rigour with a pragmatic and reusable contribution.

Objectives

General objective

To design a *backend* architecture based on the principles of distributed computing and *serverless* models, capable of shifting validation authority from the client to a controlled environment, thereby guaranteeing the integrity of the critical data of a video game.

Specific objectives

- Protect the fundamental elements of the game (licences, scores, progression) against external tampering.
- Ensure player authentication and identity through centralised and verifiable mechanisms.
- Implement validation flows that guarantee the consistency of the data sent by the client.
- Establish logs and traceability mechanisms that make it possible to detect irregularities and audit actions.
- Develop a conceptual model that may serve as a replicable reference for future projects.

These objectives aim to build a framework that prioritises trust, verifiability, and system sustainability over technical complexity.

Working hypotheses

Although hypotheses are traditionally associated with empirical or experimental approaches, in this case they are used as formulations of intent, equivalent to verifiable objectives. It is proposed that:

- An architecture that centralises validation on the server reduces the tampering of scores and progression.
- The incorporation of structured telemetry makes it possible to detect anomalous behaviour at an early stage.
- Integrating security from the design stage reduces technical debt and improves the maintainability of the project.

These statements guide the development of the work and orient the validation of its results.

Methodological approach

The work will be carried out following a structured and progressive methodology:

1. **Domain and risk analysis:** identification of the most sensitive assets, the most common threats, and recurrent insecure practices.
2. **Conceptual architecture design:** definition of the authenticate–authorise–validate–log flows and of the applicable distributed computing principles.
3. **Prototype implementation:** development of a controlled environment in which the proposed measures are evaluated.
4. **Instrumentation and observability:** incorporation of logging and telemetry mechanisms to assess coherence and behaviour.
5. **Experimental evaluation:** tests under simulated adverse scenarios that allow the effectiveness of the approach to be measured.
6. **Synthesis of results:** extraction of conclusions and formulation of a *blueprint* applicable to other contexts.

Expected results

As a result of carrying out this work, the following outcomes are expected:

- A reproducible architecture that demonstrates the feasibility of shifting validation authority to the server.
- A set of best practices applicable to educational or independent projects.
- Experimental evidence that centralised verification improves data consistency and reduces tampering risks.

- A methodological contribution that promotes the integration of security from the earliest design phases.

In this way, the aim is to contribute to a more responsible and sustainable view of video game development, one in which security ceases to be an obstacle and becomes a structural principle.

Capítulo 1. Introducción

1.1. Motivación

La industria de los videojuegos constituye uno de los ejes tractores de la economía digital contemporánea. Históricamente, ha superado en volumen de ingresos y penetración global a las industrias cinematográfica y musical de forma combinada [1]. Lo que emergió como un *hobby* doméstico a principios de la década de 1970, se ha consolidado como un fenómeno cultural, ecosocial y tecnológico, sustentado por comunidades internacionales, competiciones de alto rendimiento (*eSports*) y complejas economías virtuales.

Las alteraciones fraudulentas de puntuaciones, la inyección de *cheats*, la manipulación del estado del cliente y la distribución de copias no autorizadas representan vectores de amenaza cotidianos. El usuario legítimo confía en la equidad de la experiencia interactiva y en que su progresión se evalúe con rigor. No obstante, gran parte de la infraestructura que soporta dicha confianza resulta trivialmente explotable debido a vulnerabilidades arquitectónicas subyacentes.

Esta situación no solo daña la forma en que se juega, sino que también perjudica la reputación de los desarrolladores y las comunidades de jugadores.

Históricamente, se han desplegado diversas contramedidas para mitigar estas vulnerabilidades. Una de las primeras líneas de defensa consistió en la integración de mecanismos de *Digital Rights Management (DRM)*. Estos mecanismos se concibieron para restringir la copia y ejecución no autorizada del *software*. La evolución desde simples comprobaciones de claves de activación hasta complejos ofuscadores dinámicos como *Denuvo Anti-Tamper* evidencia una carrera armamentística incesante entre la protección de la propiedad intelectual y la ingeniería inversa.

En paralelo, ha proliferado el desarrollo de sistemas de detección de intrusiones específicos para videojuegos (*anti-cheat*). Dichas soluciones han experimentado una sofisticación notable. Mientras que en sus iteraciones iniciales se limitaban a la verificación estática en el espacio de usuario, en la actualidad operan predominantemente a nivel de *kernel* del sistema operativo (por ejemplo, *BattlEye* o *Easy Anti-Cheat*), incrementando teóricamente la fricción frente a manipulaciones de memoria.

Aun así, estos mecanismos comparten una limitación estructural: dependen en gran medida de la integridad del cliente. Por sofisticados que sean, si el punto de control sigue residiendo en el dispositivo del jugador, siempre existirá una superficie de ataque abierta. La motivación de este trabajo surge precisamente de esa debilidad: repensar dónde reside la autoridad del sistema y no depender del comportamiento legítimo del jugador.

Esta debilidad estructural se manifiesta de forma recurrente en la práctica. Un caso documentado reciente es el de febrero de 2026, cuando *MKDEV* publicó un *crack* para *Resident Evil Requiem*, protegido por *Denuvo Anti-Tamper*. Para sortear la protección, el *crack* exigía desactivar funciones de seguridad integradas por defecto en entornos *Windows*, como *Secure Boot*, *VBS*, *HVCI*, *Hyper-Visor* y la integridad de memoria de *Windows Defender*, así como habilitar la carga de *drivers* en *Test-Mode*. Esta configuración

deja al equipo expuesto a *rootkits* a nivel *kernel* no detectables por el usuario ni por el sistema operativo; en este caso concreto, el *crack* distribuido incluía además un *keylogger*. Más allá de la anécdota, el episodio ilustra un patrón general: cuando el punto de control reside en el cliente, los costes que el usuario asume para eludir la protección pueden llegar a comprometer la seguridad del propio sistema operativo.

Se postula que es posible diseñar topologías de juego intrínsecamente más seguras mediante la adopción de principios de computación distribuida y arquitecturas *serverless*¹, trasladando la autoridad de validación lógica y las decisiones de estado crítico desde el cliente hacia el *backend*. Este cambio de paradigma no solo salvaguarda la integridad de la sesión, sino que establece un marco auditable, determinista y sostenible.

Problema y brecha

El problema subyacente no reside en la carencia de primitivas criptográficas o tecnológicas, sino en el paradigma arquitectónico imperante. En numerosos desarrollos —particularmente en entornos *indie* o académicos— se incurre en una sobredependencia de la integridad del cliente: se validan de facto puntuaciones, vectores de movimiento o inventarios originados en la máquina del usuario sin una posterior reconciliación autoritativa en el servidor. Aunque este modelo *client-authoritative* acelera el ciclo de desarrollo inicial, introduce un vector de ataque masivo que compromete la viabilidad del ecosistema a largo plazo.

El resultado de esta tendencia es un ecosistema fragmentado y desigual. Los juegos acaban siendo tan seguros como el dispositivo del usuario más débil. La manipulación de datos, los *exploits* y las trampas dejan de ser casos aislados para convertirse en síntomas de una arquitectura mal planteada. Además, la dependencia de soluciones locales impide escalar o mantener sistemas de forma coherente, incrementando la deuda técnica y el riesgo operativo.

Se identifica así una brecha doble:

- **Brecha técnica:** generada por la carencia de arquitecturas de referencia (o *blueprints*) que democratizen el despliegue de soluciones *server-authoritative* en equipos con recursos limitados.
- **Brecha metodológica:** al tratar la seguridad como una etapa final del desarrollo en lugar de como un principio de diseño (*security by design*).

Este trabajo busca contribuir a cerrar esa brecha mediante la definición de un enfoque arquitectónico que priorice la verificación en servidor, la observabilidad y la consistencia de los datos, sin sacrificar la agilidad ni la calidad de la experiencia, siempre desde un entorno *serverless*.

Alcance y contexto del trabajo

El presente Trabajo Fin de Grado no pretende resolver todos los problemas de seguridad de la industria, sino abordar un área concreta y representativa: la protección

¹Modelo de ejecución en la nube donde la provisión y el escalado de la infraestructura se gestionan dinámicamente por el proveedor, abstrayendo la capa física.

de los elementos críticos de un videojuego mediante un *backend*² seguro y escalable.

El alcance se delimita en torno a los siguientes ejes:

- Se centra en el diseño y evaluación de una arquitectura que gestione licencias, progresión, puntuaciones y autenticación desde el servidor.
- No se desarrolla un sistema *anti-cheat* a nivel de *kernel*, aunque su relevancia se reconoce como parte del contexto histórico.
- No se diseñan nuevos algoritmos de *DRM*, pero se analizan como antecedentes del control de integridad del cliente.
- No se abordan redes multijugador en tiempo real ni aspectos de latencia, ya que el foco recae en la seguridad lógica y la consistencia del estado del juego.

Desde el punto de vista contextual, este trabajo se inscribe en dos marcos complementarios:

- **Académico y formativo:** al tratarse de un proyecto orientado al aprendizaje aplicado de ciberseguridad, arquitectura de sistemas y desarrollo de videojuegos.
- **Práctico y replicable:** al ofrecer un modelo viable para estudios independientes o proyectos de pequeña escala, donde la seguridad suele quedar relegada por limitaciones de tiempo o recursos.

Se considera que el valor de esta investigación reside precisamente en esa doble perspectiva: combinar el rigor académico con una aportación pragmática y reutilizable.

1.2. Objetivos

Objetivo general

Diseñar una arquitectura de *backend* basada en principios de computación distribuida y modelos *serverless* que permita trasladar la autoridad de validación del cliente a un entorno controlado, garantizando la integridad de los datos críticos de un videojuego.

Objetivos específicos

- Proteger los elementos fundamentales del juego (licencias, puntuaciones, progresión) frente a manipulaciones externas.
- Asegurar la autenticación y la identidad de los jugadores mediante mecanismos centralizados y verificables.
- Implementar flujos de validación que garanticen la coherencia de los datos enviados por el cliente.
- Establecer registros y trazabilidad que permitan detectar irregularidades y auditar acciones.

²Parte de una aplicación o sitio web que permanece invisible para el usuario final.

- Desarrollar un modelo conceptual que sirva como referencia replicable para futuros proyectos.

Estos objetivos se orientan a construir un marco que priorice la confianza, la verificabilidad y la sostenibilidad del sistema por encima de la complejidad técnica.

1.3. Hipótesis de trabajo

Aunque tradicionalmente las hipótesis se asocian con enfoques empíricos o experimentales, en este caso se emplean como formulaciones de intención, equivalentes a objetivos verificables. Se plantea que:

- Una arquitectura que centraliza la validación en servidor reduce la manipulación de puntuaciones y progresión.
- La incorporación de telemetría estructurada permite detectar comportamientos anómalos de forma temprana.
- La integración de la seguridad desde el diseño disminuye la deuda técnica y mejora la mantenibilidad del proyecto.

Estas afirmaciones guían el desarrollo del trabajo y orientan la validación de los resultados.

1.4. Enfoque metodológico

El trabajo se desarrollará siguiendo una metodología estructurada y progresiva:

1. **Análisis del dominio y riesgos:** identificación de los activos más sensibles, las amenazas comunes y las prácticas inseguras recurrentes.
2. **Diseño conceptual de arquitectura:** definición de los flujos autenticar–autorizar–validar–registrar y de los principios de computación distribuida aplicables.
3. **Implementación de un prototipo:** desarrollo de un entorno controlado donde se evalúen las medidas propuestas.
4. **Instrumentación y observabilidad:** incorporación de mecanismos de registro y telemetría para evaluar coherencia y comportamiento.
5. **Evaluación experimental:** pruebas con escenarios adversos simulados que permitan medir la eficacia del enfoque.
6. **Síntesis de resultados:** obtención de conclusiones y formulación de un *blueprint* aplicable a otros contextos.

1.5. Resultados esperados

Como resultado del desarrollo de este trabajo, se espera obtener:

- Una arquitectura reproducible que demuestre la viabilidad de trasladar la autoridad de validación al servidor.
- Un conjunto de buenas prácticas aplicables a proyectos educativos o independientes.
- Evidencia experimental de que la verificación centralizada mejora la coherencia de los datos y reduce los riesgos de manipulación.
- Una aportación metodológica que promueva la integración de la seguridad desde las primeras fases de diseño.

De este modo, se pretende contribuir a una visión más responsable y sostenible del desarrollo de videojuegos, en la que la seguridad deje de ser un obstáculo para convertirse en un principio estructural.

Capítulo 2. Estado de la Cuestión

2.1. Introducción

Los videojuegos modernos dependen de infraestructuras cada vez más complejas y distribuidas. Detrás de cada partida multijugador, cada sistema de progresión o cada evento en vivo existe un entramado técnico que combina motores de juego, servicios en la nube, redes de distribución de contenido y bases de datos globales. Esta transformación ha permitido experiencias en tiempo real para millones de usuarios, pero también ha multiplicado la superficie de ataque.

El paso del entretenimiento local al entorno conectado ha cambiado las reglas del desarrollo. Hoy, el cliente ya no es un ejecutable aislado, sino una puerta de entrada a ecosistemas persistentes que gestionan identidades, economías virtuales y datos sensibles. Esta hiperconectividad ha hecho que la seguridad deje de ser un añadido opcional para convertirse en un principio de diseño estructural.

El objetivo de este capítulo es examinar cómo la industria del videojuego ha abordado la ciberseguridad desde distintos enfoques —históricos y contemporáneos—, identificando las limitaciones de los modelos tradicionales basados en la confianza del cliente y analizando las oportunidades que ofrecen las arquitecturas distribuidas y *serverless*. Asimismo, se presentan los marcos teóricos y normativos que orientan las buenas prácticas de seguridad en entornos de nube, junto con las tendencias emergentes que buscan integrar la seguridad como un componente verificable y automatizado.

2.2. Riesgos y vulnerabilidades en videojuegos conectados

El ecosistema ludológico interconectado contemporáneo hereda sustanciales vulnerabilidades de las aplicaciones web y móviles, pero amplificadas por la interactividad en tiempo real y por la exposición global de sus servicios. La arquitectura cliente-servidor clásica, ampliamente adoptada desde los años noventa, parte de una suposición insegura: el cliente se comportará honestamente. En la práctica, esta confianza se traduce en múltiples puntos de ataque.

Manipulación del cliente

La manipulación del cliente incluye técnicas como la modificación de memoria, la inyección de bibliotecas dinámicas (*DLL¹ injection*), el parcheo del ejecutable o el uso de herramientas de depuración para alterar la lógica del juego. Estos ataques permiten modificar variables internas, acelerar movimientos, alterar físicas o incluso desactivar comprobaciones locales.

En títulos competitivos como *Counter-Strike: Global Offensive* o *Fortnite*, las trampas basadas en *memory editing* y *aimbots* se han convertido en un problema estructural. Aunque los sistemas *anti-cheat* intentan detectar estas alteraciones, la constante actualización del software y la creatividad de los atacantes generan una carrera

¹Dynamic Link Library, término con el que se refiere a los archivos con código ejecutable que se cargan bajo demanda de un programa por parte del sistema operativo.

interminable.

Estudios recientes como el de Prazak et al. [2] demuestran que la telemetría puede ser una herramienta eficaz para detectar anomalías en el comportamiento del jugador, mediante análisis estadístico y aprendizaje automático. Sin embargo, este enfoque suele ser reactivo y requiere una arquitectura que permita recopilar y procesar grandes volúmenes de datos en tiempo real.

Abuso de APIs y comunicación

Otra fuente crítica de vulnerabilidades son las APIs² de *backend*³. Si no se implementan controles adecuados, estas interfaces pueden ser explotadas para alterar puntuaciones, progresos o inventarios. Casos documentados en comunidades de juegos móviles y plataformas independientes muestran cómo herramientas como *Fiddler* o *Charles Proxy* permiten interceptar tráfico HTTP⁴ y modificar solicitudes antes de que lleguen al servidor.

Según la OWASP⁵ Foundation [3], las vulnerabilidades más comunes en APIs incluyen autenticación deficiente, exposición excesiva de datos, falta de límites de petición y errores de autorización a nivel de objeto. Estas debilidades son especialmente graves en entornos de juego, donde los *endpoints*⁶ suelen manejar información económica o de progresión persistente.

Gestión deficiente de identidad y sesión

En los videojuegos modernos, las identidades digitales representan activos valiosos. Cuentas comprometidas pueden dar acceso a compras, logros y progresos. La reutilización de contraseñas, la falta de doble autenticación y los *tokens*⁷ sin caducidad son problemas recurrentes.

Servicios como *Cognito* [4] y los esquemas basados en OAuth 2.0 y JWT⁸, ofrecen mecanismos robustos de autenticación federada y control de sesión.

Riesgos operativos

Más allá del plano técnico, los videojuegos modernos enfrentan riesgos operativos derivados de su naturaleza dinámica. Las actualizaciones en vivo, los parches rápidos y los eventos temporales modifican constantemente la superficie de ataque. Una regla de acceso mal configurada en un despliegue puede dejar expuestos datos sensibles o APIs

²Application Programming Interface, conjunto de reglas, protocolos y herramientas que permite que diferentes aplicaciones de software se comuniquen entre sí e intercambien datos de manera estandarizada y definida.

³Parte oculta de una aplicación web o software que se ejecuta en el servidor.

⁴Hypertext Transfer Protocol, protocolo fundamental de la World Wide Web que define las reglas para la comunicación entre los navegadores web y los servidores.

⁵Open Web Application Security Project, proyecto de código abierto dedicado a determinar y combatir las causas que hacen que el software sea inseguro.

⁶Punto donde un sistema expone una funcionalidad a la que otro sistema puede acceder de forma remota.

⁷Mecanismo digital que verifica la identidad de un usuario o dispositivo sin necesidad de usar credenciales tradicionales repetidamente.

⁸JSON Web Token, formato estándar (RFC 7519) para transmitir información autenticada entre dos partes mediante un objeto JSON firmado digitalmente.

internas. La seguridad, en este contexto, no puede entenderse como un estado, sino como un proceso continuo de revisión, auditoría y observabilidad.

2.3. Estrategias tradicionales de protección

A lo largo de las últimas tres décadas, la industria del videojuego ha desarrollado diversas estrategias para proteger su software y su integridad. Las más destacadas son los sistemas de *Digital Rights Management (DRM)* y los sistemas *anti-cheat*, que aunque diferentes en propósito, comparten una dependencia común del entorno del cliente.

Gestión de derechos digitales (DRM)

Los mecanismos de *DRM* surgieron como respuesta directa a la piratería y la distribución no autorizada. Desde los clásicos seriales de activación de los años noventa hasta los modernos esquemas de validación en línea, el *DRM* ha intentado asegurar que cada copia del juego sea legítima.

Ejemplos tempranos incluyen el sistema de activación de *Microsoft Games for Windows Live* o las validaciones por *CD-Key* en *Blizzard*. Con la expansión de las plataformas digitales —*Steam*, *Origin* o *Epic Games Store*—, el *DRM* pasó a formar parte del ecosistema, integrando la verificación en línea como requisito de ejecución.

El caso más emblemático es *Denuvo Anti-Tamper* [5], diseñado para impedir la ingeniería inversa y proteger el ejecutable. Aunque eficaz para retrasar la piratería, ha generado debate por su impacto en el rendimiento y por las implicaciones en la experiencia del usuario [6]. El *DRM* ilustra el dilema clásico entre control y accesibilidad: cuanto más restrictivo es el sistema, mayor es la fricción con el usuario final.

Sistemas *anti-cheat*

Los sistemas *anti-cheat* tienen como objetivo preservar la equidad en entornos competitivos. Herramientas como *BattlEye* y *Easy Anti-Cheat* [7] operan a bajo nivel del sistema operativo, monitorizando procesos, llamadas de sistema y firmas de código para detectar trampas.

Existen dos enfoques principales: (1) la detección heurística, basada en reglas predefinidas y listas negras, y (2) la detección conductual, apoyada en análisis de telemetría y aprendizaje automático.

Los sistemas *kernel-level*, como los empleados por *Valorant* o *Fortnite*, ofrecen control profundo pero plantean serios desafíos de privacidad, compatibilidad y mantenimiento. Además, siguen compartiendo el problema de base: dependen de un cliente que puede ser manipulado.

2.4. Evolución de topologías de red en videojuegos

La evolución de las topologías de red en videojuegos refleja un traslado progresivo de la autoridad desde los clientes hacia infraestructuras más controladas. Cada salto reduce determinados vectores de ataque, aunque introduce nuevas responsabilidades operativas y de observabilidad.

La Figura 2.1 sintetiza esta evolución y compara las implicaciones de seguridad de cada modelo.

Peer-to-Peer Clientes conectados entre sí. Autoridad distribuida. Riesgos: Manipulación local. Suplantación entre pares. Desincronización.	Cliente-servidor dedicado Servidor central persistente. Autoridad de partida. Mitiga: Estado no autoritativo. Tráfico P2P manipulable. Fraude entre clientes.	Arquitectura <i>serverless</i> Servicios gestionados. Validación bajo demanda. Mitiga: Administración manual. Escalado rígido. Falta de trazabilidad.
---	--	--

Figura 2.1: Comparativa entre topología *Peer-to-Peer*, cliente-servidor dedicada y arquitectura *serverless*.

La computación en la nube ha alterado de raíz la forma de plantear el *backend* de un videojuego. Frente a las infraestructuras estáticas de antaño, hoy resulta natural recurrir a plataformas que escalan recursos bajo demanda. Suelen distinguirse tres modelos:

- *IaaS (Infrastructure as a Service)*: el proveedor ofrece máquinas virtuales y almacenamiento, y el usuario se ocupa del sistema operativo y las aplicaciones.
- *PaaS (Platform as a Service)*: el entorno de ejecución viene ya gestionado, de modo que basta con desplegar la aplicación.
- *FaaS (Function as a Service)*: también conocido como modelo *serverless*, ejecuta el código en respuesta a eventos y solo factura por las invocaciones realizadas.

En el caso de los videojuegos, el enfoque *serverless* encaja especialmente bien con el objetivo de mover la autoridad del cliente al servidor: validar progresos, licencias o puntuaciones en un entorno fuera del alcance del jugador. *Lambda* [8] se encarga de la lógica de negocio, *API Gateway* [9] actúa como puerta de entrada, con autenticación y control de tráfico, *Cognito* [4] gestiona identidad y sesión, y *DynamoDB* [10] ofrece el almacenamiento subyacente.

Al no exponer servidores propios a Internet, la superficie de ataque se reduce de forma considerable. La configuración y revisión de la seguridad del sistema se ha apoyado en referencias como el *CIS AWS⁹ Foundations Benchmark* [11] y el modelo de *Zero Trust* [12]. AWS es la plataforma en la nube sobre la que se construye el proyecto, y proporciona los servicios de cómputo, almacenamiento, bases de datos y red empleados en la arquitectura.

El modelo *serverless* no está, sin embargo, exento de inconvenientes: la trazabilidad de eventos se complica, aparece una fuerte dependencia del proveedor y la observabilidad pasa a ser un requisito de primer orden. De ahí la importancia de las herramientas de

⁹*Amazon Web Services*, plataforma de computación en la nube que ofrece servicios de cómputo, almacenamiento, bases de datos y red bajo demanda.

auditoría: *CloudWatch*¹⁰, encargado de recopilar métricas, *logs* y eventos tanto de los recursos de *AWS* como de las aplicaciones desplegadas sobre ellos, y *AWS X-Ray*, que permite seguir el rastro de cada petición a lo largo de la arquitectura.

2.5. Marcos de seguridad y principios de diseño

El diseño seguro de sistemas distribuidos se apoya en principios teóricos consolidados que orientan la implementación práctica de medidas de protección.

Zero Trust

El paradigma de *Zero Trust*, definido por el *NIST* [12], establece que ningún usuario, servicio o dispositivo debe considerarse confiable por defecto. Cada solicitud debe autenticarse, autorizarse y verificarse según el contexto. Aplicar este principio a videojuegos implica validar cada acción enviada desde el cliente, asegurando que el servidor mantenga siempre la autoridad sobre el estado del juego.

Seguridad por diseño y *DevSecOps*

El enfoque de *seguridad por diseño* promueve la integración temprana de la ciberseguridad en el ciclo de desarrollo. En el ámbito del videojuego, esto significa incluir validaciones, registros y controles de identidad desde la fase de prototipado.

Las metodologías *DevSecOps* amplían este concepto incorporando automatización de pruebas, análisis estático de código y despliegues controlados con verificación continua. Plataformas como *AWS CodePipeline* o *GitHub Actions* permiten integrar escaneos de seguridad y auditorías antes de cada despliegue, reduciendo la probabilidad de errores humanos.

Normativas y estándares

Organizaciones como la *OWASP* y el *Center for Internet Security (CIS)* [11, 13] ofrecen marcos de referencia ampliamente aceptados. La adopción de controles de configuración y auditoría permite alinear las arquitecturas con estándares de la industria. En particular, los *CIS Controls v8* establecen prácticas fundamentales como la gestión de identidades, el registro centralizado y la minimización de privilegios.

2.6. Brecha identificada y tendencias emergentes

Pese a la existencia de herramientas, guías y principios consolidados, persiste una brecha significativa entre teoría y práctica. En entornos académicos o de desarrollo independiente, la seguridad suele verse como un obstáculo o un coste añadido, lo que resulta en proyectos que priorizan la jugabilidad sobre la robustez del *backend*.

Esta brecha se manifiesta en dos niveles: (1) una brecha técnica, relacionada con la ausencia de marcos reproducibles que faciliten la adopción de seguridad en la nube; y (2) una brecha metodológica, vinculada a la falta de cultura de seguridad integrada en el desarrollo.

¹⁰Servicio de monitorización y observabilidad de *AWS* que recopila y rastrea métricas, *logs* y eventos de recursos de *AWS*, aplicaciones en la nube y entornos locales.

La seguridad de los videojuegos se dirige hoy hacia un modelo de hibridación. Títulos a gran escala como *Valorant* y *Overwatch* ya combinan el análisis de telemetría [2] con aprendizaje automático para detectar acciones atípicas en el cliente y en el servidor. Todos los indicios apuntan a que la seguridad dejará de ser reactiva y pasará a depender de lo observable y verificable mediante arquitecturas *serverless*.

El trabajo se posiciona, por tanto, sobre los dos cuadrantes menos sustituibles: la **validación *server-side*** y la **observabilidad auditada**. No pretende competir con el *DRM* ni con el *anti-cheat* de cliente, sino cubrir lo que ninguno de ellos resuelve de raíz: que el estado autorizado del juego (licencias, sesiones, puntuaciones, progresión) sea íntegro y auditable. Por el principio extremo a extremo [14], esa autoridad solo puede ejercerse en el servidor. La contrapartida, asumida desde el principio, es que las trampas puramente perceptivas, quedan fuera del alcance del trabajo y se tratan como un problema aparte.

En este sentido, el trabajo no se plantea como un sustituto de los modelos existentes, sino como una pieza que llena un hueco concreto en el mapa: ofrecer un *blueprint* reproducible, basado en *serverless* sobre *AWS*, que centralice autenticación, validación y trazabilidad, y que resulte accesible para proyectos independientes.

Capítulo 3. Tecnologías utilizadas y Metodología

3.1. Visión general de la arquitectura

El prototipo se estructura como un *backend serverless* para un videojuego desarrollado en *Unity* y que se acompaña de un portal *web* de autoservicio para el registro y la gestión de licencias. La adopción del modelo *serverless* no responde a una preferencia tecnológica genérica, sino a dos restricciones concretas del proyecto: aislar en un entorno controlado la lógica sensible, control del modo *demo* y verificación de integridad del ejecutable.

El cliente *Unity* (C#) y la página *HTML/JS* de gestión de licencias comparten un mismo *user pool* de *Cognito* y se comunican con el *backend* a través de una *API HTTP* expuesta por *API Gateway*, que valida en el perímetro los *tokens JWT* emitidos por *Cognito* antes de delegar la petición en una función *Lambda*. Cada *Lambda* implementa una operación concreta del dominio y persiste su estado en tablas independientes de *DynamoDB*. El ejecutable del juego se sirve desde *S3* y *CloudWatch* centraliza los registros y las métricas sobre las que se apoya la observabilidad del sistema.

La justificación detallada de cada elección se aborda en la sección siguiente.

3.2. Justificación de tecnologías

3.2.1. Capa de entrada: *API Gateway*

API Gateway funciona como punto de entrada del sistema: recibe las peticiones del cliente *Unity* y del portal *web* y las redirige a la *Lambda* que corresponde.

Se ha optado por este servicio por tres motivos. Su integración con *Cognito* permite validar los *tokens JWT* en el propio perímetro, comprobando firma, expiración y emisor antes de que la petición llegue al *backend*; si algo no encaja, se descarta sin invocar ninguna *Lambda*. Además, cada método admite un esquema *JSON*¹ como contrato de entrada, lo que filtra las solicitudes mal formadas desde el principio y ahorra tener que repetir esa validación en cada función. Por último, los límites configurables por *endpoint*, identidad o *IP*² protegen las operaciones más sensibles, como el envío de puntuaciones o la verificación del ejecutable, sin necesidad de componentes externos.

3.2.2. Capa de lógica: *Lambda*

La lógica del *backend* se divide en varias funciones *Lambda*, cada una con una responsabilidad propia: registrar sesiones, validar estadísticas, comprobar el *hash*³ del ejecutable, controlar el modo *demo*, emitir licencias y generar *URLs*⁴ prefiradas. Cada

¹JavaScript Object Notation, formato ligero de intercambio de datos basado en pares clave-valor.

²Internet Protocol, dirección numérica que identifica a cada dispositivo conectado a una red.

³Cadena de longitud fija obtenida al aplicar una función matemática a un dato de entrada; sirve para verificar la integridad del dato, ya que cualquier modificación produce un valor distinto.

⁴Uniform Resource Locator, cadena que identifica unívocamente un recurso accesible a través de la web.

endpoint de *API Gateway* llama a una sola función, así que los permisos pueden ajustarse caso por caso: una *Lambda* solo accede a las tablas de *DynamoDB* y a los *buckets* de *S3* que le hacen falta.

Como las funciones *Lambda* se ejecutan sin estado, las invocaciones son independientes entre sí. Esto evita que un atacante pueda mantener un proceso activo entre peticiones y, si una invocación falla, las demás no se ven afectadas.

3.2.3. Capa de datos: *DynamoDB*

El estado del prototipo se guarda en varias tablas de *DynamoDB*, una por dominio: perfil y progreso del jugador, sesiones, control del modo *demo*, verificación de integridad del ejecutable y registro de eventos. Las consultas que necesita el sistema son sencillas —búsquedas por identificador de usuario, de sesión o de *hash*—, así que un modelo clave-valor cubre los casos del prototipo sin tener que recurrir a un motor relacional como *Aurora*, más caro de mantener e incongruente a esta escala.

Otra ventaja útil es el atributo *TTL*⁵, que permite borrar automáticamente los datos efímeros, como *nonces* de un solo uso o *tokens* caducados, sin tener que añadir tareas de limpieza periódicas.

3.2.4. Capa de almacenamiento y distribución: *S3*

S3 se usa como almacén del ejecutable del juego tanto en *macOS* como en *Windows*. El *bucket* no expone los objetos directamente: cada descarga pasa por una *URL* prefirmada que genera una *Lambda* y que caduca tras un tiempo corto, así que el contenido nunca queda accesible públicamente y no hace falta distribuir credenciales desde el cliente.

El control de acceso encaja con esta forma de trabajar. Solo las funciones que generan los enlaces tienen permiso de lectura sobre los objetos; el cliente *Unity* y el resto del sistema acceden únicamente a través de las *URLs* firmadas.

3.2.5. Capa de identidad: *Cognito*

La gestión de identidades se delega en *Cognito* mediante un único *user pool* que comparten el cliente *Unity* y el portal web. El *backend* no llega a almacenar contraseñas: la autenticación pasa íntegramente por *Cognito*, que utiliza el protocolo *SRP*⁶ para que las credenciales no viajen en claro.

Una vez autenticado, el usuario recibe tres *tokens JWT* firmados —*access token*, *ID token* y *refresh token*— que *API Gateway* valida en cada petición. El *access token* caduca pronto, así que es responsabilidad del cliente realizar la petición de la renovación en segundo plano, usando el *refresh token*, sin volver a pedir credenciales al jugador. Si se detecta actividad sospechosa, basta con revocar el *refresh token* para cortar el acceso de una sesión.

⁵Time To Live, mecanismo que asigna un límite de tiempo a los datos, transcurrido el cual se eliminan automáticamente

⁶Secure Remote Password, protocolo criptográfico para la autenticación de contraseñas sin transmitir las en claro.

3.2.6. Capa de observabilidad: *CloudWatch*

CloudWatch centraliza los *logs* de las funciones *Lambda* y las métricas operativas que el prototipo utiliza para conocer su estado en cada momento. Las funciones emiten registros estructurados que permiten reconstruir el recorrido de cada petición desde su entrada por *API Gateway* hasta su persistencia en *DynamoDB*.

De esos registros se derivan las métricas que se visualizan en los paneles: latencias *p50* y *p95*, contadores de errores 4XX y 5XX, peticiones por segundo por *endpoint* y validaciones fallidas por esquema *JSON* o *token*.

3.3. Diseño de datos y contratos

3.3.1. Modelo de datos

En lugar de centralizar todo el estado en una única tabla, el diseño separa la información en varias tablas independientes de *DynamoDB*: perfil y progreso del jugador, sesiones activas, control del modo demo, verificación de integridad del ejecutable y registro de eventos. Esta separación clarifica las responsabilidades y permite que cada *Lambda* acceda solo al subconjunto de datos que necesita.

Cada tabla se modela en torno a sus consultas reales. La de sesiones se consulta por identificador de sesión, la del modo demo registra marcas de tiempo y tiempo acumulado por usuario, y la de *hashes* guarda los valores que permiten validar la versión que ejecuta el cliente.

3.3.2. Esquema de eventos

Las funciones *Lambda* registran cada acción con una estructura común. Algunos de los campos principales son:

- **timestamp**: cuándo ocurrió el evento, en milisegundos.
- **userID**: usuario que la realizó, según la identidad de *Cognito*.
- **IP**: dirección desde la que se hizo la petición.
- **deviceID**: dispositivo desde el que se conectó.

Cada evento se clasifica además por nivel de severidad:

- **REPORT**: diagnóstico para el desarrollo (RequestID, tiempo de ejecución, memoria).
- **INFO**: funcionamiento normal del sistema.
- **WARNING**: situaciones que requieren atención, como fallos repetidos de autenticación.
- **ERROR**: fallos que afectan a una funcionalidad concreta sin detener el sistema.

3.3.3. Contratos de API

El *backend* expone un conjunto reducido de *endpoints*, cada uno asociado a una operación concreta del juego:

- GET /get-game: información base para iniciar la aplicación.
- POST /register-session: registro de una nueva sesión de juego.
- POST /verify-stats: validación de las estadísticas enviadas por el cliente.
- POST /check-version: comprobación de la versión activa del juego.
- GET /demoCheck: estado temporal del acceso en modo demo.
- ANY /VerifyGameHash: validación de la integridad de la versión ejecutada.

Cada *endpoint* lleva asociado un esquema *JSON Schema*⁷ que define los campos obligatorios, los tipos esperados y las restricciones de cada petición. Las solicitudes que no encajan con su contrato se rechazan en *API Gateway*, antes de invocar a la *Lambda* correspondiente.

Las respuestas siguen una estructura unificada: el código *HTTP* refleja el estado general de la conexión y el cuerpo del mensaje añade un código de negocio que el cliente *Unity* utiliza para distinguir situaciones específicas. Así el juego puede reaccionar a cada caso sin tener que interpretar los códigos *HTTP* de forma ambigua.

3.4. Flujos críticos del sistema

3.4.1. Alta de usuario y gestión de identidad

La gestión de usuarios comienza en una web de registro que comparte el mismo *user pool* de *Cognito* que el resto del sistema. El proceso es intuitivo para el jugador: crea su cuenta introduciendo sus datos básicos y el sistema se encarga automáticamente de validar que las credenciales sean correctas y que no existan registros duplicados.

Tras confirmar la identidad mediante un código de verificación enviado por correo electrónico, esa cuenta se convierte en la llave única del usuario. Con esta misma identidad, el jugador podrá iniciar sesión de forma fluida tanto en el portal web como en el juego, lo que permite que el *backend* reconozca y gestione cada perfil de manera individual y segura en todo momento.

A partir de ese momento, la aplicación trabaja sobre una identidad ya autenticada, lo que permite que el resto de funciones del *backend* operen sobre un identificador único y verificado por *Cognito*. De este modo, la gestión de licencias no depende de un alta manual adicional, sino de la existencia de una cuenta válida y activada en el sistema de autenticación centralizado.

⁷Vocabulario que permite la coherencia, validez e interoperabilidad de los datos *JSON* a gran escala.

3.4.2. Envío de puntuaciones

El envío de puntuaciones cuenta con una serie de medidas para evitar abusos y garantizar la protección del sistema.

Con respecto a la coherencia de los datos, el servidor valida que las puntuaciones enviadas tienen sentido. Se compara con el progreso del usuario o se busca que no haya grandes saltos, se trata de que no se encuentren valores atípicos o manipulados.

Todos los intentos que incumplen los puntos tocados anteriormente quedan registrados en *logs* de auditoría. Estas infracciones pueden hacer que salten alarmas de forma automática para que se lleve a cabo una revisión manual de estos casos y así tener una respuesta temprana ante estos casos fraudulentos.

3.5. Seguridad y cumplimiento

3.5.1. Zero Trust

El *backend* asume que ninguna petición es de confianza por defecto, ni siquiera las que vienen del propio sistema: cada acceso se autentica y se autoriza explícitamente. La autenticación la cubre el flujo de *Cognito* y *API Gateway* descrito antes; lo que añade *Zero Trust* son las restricciones de qué puede hacer cada usuario una vez autenticado.

Esto se materializa mediante *IAM policies*⁸, que especifican quién puede acceder a qué recurso y con qué acciones. En la práctica, un jugador solo puede consultar sus propias puntuaciones y no las de otros, y cada *Lambda* accede únicamente a las tablas y *buckets* que necesita para su tarea. [15]

3.5.2. Endurecimiento de superficie

Para que el tráfico abusivo no llegue a la lógica del juego, el sistema combina varias capas de filtrado en el perímetro:

- *WAF*⁹: un *firewall* de aplicaciones web inspecciona cabeceras, cuerpos y rutas, y bloquea patrones de ataque conocidos antes de que la petición pase a *API Gateway*.
- *Throttling*¹⁰: límites de peticiones por *IP*, usuario y *token*, ajustados a nivel global y por *endpoint* crítico (envío de puntuaciones, autenticación).
- Restricciones por origen: solo se aceptan peticiones desde orígenes y agentes de usuario esperados; el resto se descarta como fricción adicional para clientes no legítimos.
- Validación de esquema: cada método aplica un contrato *JSON* estricto y rechaza en el perímetro las solicitudes con campos faltantes, tipos incorrectos o estructuras inesperadas.

⁸Identity and Access Management policies, documentos en *JSON* que definen qué acciones puede realizar una identidad sobre los recursos de *AWS*.

⁹Web Application Firewall, es una herramienta de seguridad que protege sitios y aplicaciones web filtrando y monitorizando el tráfico *HTTP* entre la aplicación e Internet.

¹⁰Técnica de gestión de tráfico que limita el número de solicitudes que un usuario o aplicación puede realizar en un periodo de tiempo.

3.5.3. Gestión de secretos

El prototipo no incorpora un servicio dedicado de gestión de secretos como *Secrets Manager* o *Parameter Store*. En su lugar, la estrategia se apoya en minimizar la exposición: *Cognito* asume la autenticación, las funciones *Lambda* operan con *tokens* temporales y permisos *IAM* mínimos, y los objetos privados de *S3* se sirven mediante *URLs* prefirmadas en lugar de exponer credenciales del *bucket*.

La lógica de validación y firma se mantiene íntegramente en el *backend*, de modo que ninguna clave ni valor sensible viaja al cliente *Unity* ni queda visible en el código distribuido.

3.5.4. Trazabilidad/Auditoría

La trazabilidad del sistema se orienta a registrar y correlacionar las acciones relevantes del *backend*, de forma que sea posible reconstruir el comportamiento de una petición desde su entrada hasta su persistencia. En lugar de plantear un sistema de auditoría forense completo, la arquitectura prioriza la observabilidad operativa: las peticiones llegan a través de *API Gateway*, son procesadas por funciones *Lambda* y dejan rastro en *CloudWatch* mediante registros estructurados que facilitan la depuración, el seguimiento de errores y la detección de comportamientos anómalos.

Además de los registros de ejecución, parte del estado relevante del dominio queda almacenado en las tablas de *DynamoDB*, lo que permite verificar posteriormente decisiones como validaciones de licencias, comprobaciones de versión, control de *demo* o rechazo de estadísticas inconsistentes. Esta combinación de registros de aplicación y persistencia de estado proporciona evidencias suficientes para revisar incidencias concretas y entender qué decisión tomó el sistema en cada momento, sin necesidad de mantener un subsistema específico de auditoría separado.

3.6. Metodología

La metodología combina iteraciones funcionales con controles de seguridad integrados desde el diseño. La Figura 3.1 resume cómo se conectan diseño, código, pruebas, despliegue, telemetría y auditoría.

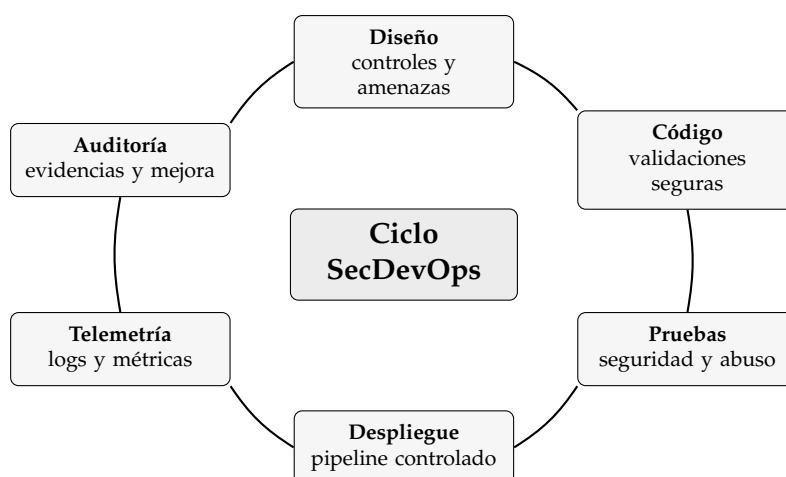


Figura 3.1: Integración de procesos de validación de seguridad en el ciclo de desarrollo ágil.

3.6.1. Enfoque incremental

El *backend* se ha desarrollado de forma incremental: cada iteración añade una capa de funcionalidad y de seguridad sobre la anterior. Se han priorizado los flujos críticos —autenticación, validación de puntuaciones, verificación de integridad del ejecutable, control del modo *demo* y telemetría— y el resto se ha ido incorporando en ciclos sucesivos. Así se reduce el riesgo y se validan pronto las decisiones de diseño, dejando margen para corregirlas cuando hace falta.

Cada incremento da lugar a un sistema funcional, que se lleva a un entorno de pruebas y se somete a ejercicios de validación y observabilidad. Si en ese punto aparecen amenazas o restricciones imprevistas, la arquitectura se retoca antes de pasar al siguiente paso. La idea de fondo es la de *security by design*: la seguridad no se añade al final del proceso, sino que acompaña al desarrollo desde el principio [16].

3.7. Consideraciones de privacidad y ética

La privacidad se ha tenido en cuenta desde el inicio del diseño. Como el *backend* maneja identidades y telemetría, conviene tener claro qué se almacena, cuánto tiempo y para qué. El prototipo se apoya en tres ideas:

- **Lo que se recoge.** Solo lo imprescindible: *ID* del usuario, puntuación, *IP* para auditoría y huella del dispositivo. No se guardan nombres reales ni información de pago.
- **Cuánto tiempo se conserva.** Los datos efímeros, como los *tokens* de sesión y los *nonces*, se borran solos gracias al *TTL* de *DynamoDB*, así que no quedan registros más allá del tiempo necesario.
- **Para qué se usa.** La telemetría sirve para detectar trampas y mantener el juego estable, nada más. No se emplea para perfilar al jugador ni para ajustar la dificultad con fines económicos.

3.8. Conclusiones del capítulo

En este capítulo se ha argumentado y fundamentado que una arquitectura *serverless* sobre AWS constituye la alternativa más adecuada para responder tanto a los requisitos de seguridad como a las necesidades de escalado del sistema. El uso de *API Gateway*, *Lambda*, *DynamoDB*, *Cognito*, *S3*, y *CloudWatch* nos brinda la posibilidad de centralizar la lógica en el *backend*, pudiendo aislar el código y los elementos sensibles. De esta forma, se aplican los principios de *Zero Trust* manteniendo un registro continuo de las acciones del jugador y de la plataforma. Además se ha incidido en que la seguridad no es un añadido posterior, sino que es un principio rector que impregna todas las capas.

La adopción de Infraestructura como Código mediante plantillas *AWS Serverless Application Model* garantiza además que la arquitectura sea reproducible y auditable, constituyendo un punto de partida sólido sobre el que escalar el *backend* de forma controlada y sin configuraciones manuales propensas a errores.

Capítulo 4. Arquitectura y Diseño

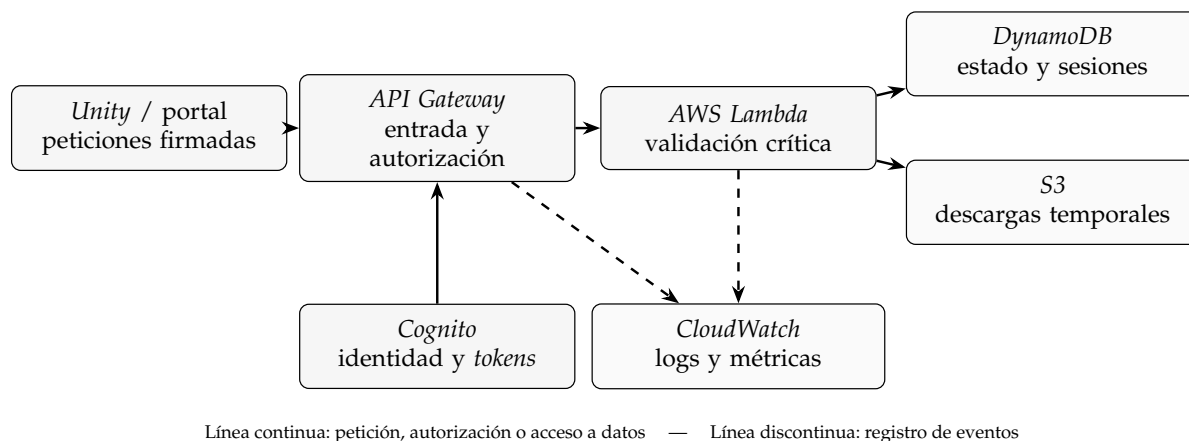


Figura 4.1: Topología de componentes *cloud* para la validación transaccional.

4.1. Resumen de la arquitectura y capas

Como se anticipa en la Figura 4.1, este capítulo describe la arquitectura *backend* propuesta en los anteriores capítulos, que traslada toda la autoridad de validación desde *Unity* hacia un entorno *serverless* en AWS. La frontera de confianza se sitúa estrictamente en el *API Gateway*, el cliente simplemente se limita a enviar los datos firmados, pero no realiza ninguna operación y mucho menos las operaciones críticas. Se organiza en capas bien definidas, que ya se definen en el capítulo 3, capa de entrada (*API Gateway*), capa de identidad (*Cognito*), lógica (*Lambda*), datos (*DynamoDB*), distribución (*S3*) y observabilidad y monitorización (*CloudWatch*), alineadas con los objetivos planteados en el capítulo 1: coherencia de datos, trazabilidad mediante *logging* y escalabilidad *serverless*, y agilidad sin comprometer la agilidad del desarrollo.

4.1.1. Descomposición por capas

La arquitectura se estructura en las siguientes capas funcionales:

Esta descomposición de capas mantiene los principios de *Zero Trust* en el punto de entrada, reduce la superficie de ataque al encapsular el acceso a servicios externos, negando cualquier operación por parte del cliente y facilita la incorporación de nuevos módulos sin reescribir el *backend*.

Capa	Servicio AWS	Responsabilidad Principal
Entrada	<i>API Gateway</i>	Exposición segura de <i>endpoints</i> y autorización inicial
Identidad	<i>Cognito</i>	Autenticación federada y gestión de sesiones/ <i>tokens</i>
Lógica	<i>Lambda</i>	Validación de <i>hashes</i> , estadísticas y lógica de negocio crítica
Datos	<i>DynamoDB</i>	Almacenamiento coherente de progresos, sesiones y telemetría
Distribución	S3	Descarga segura de assets con enlaces temporales pre-firmados
Observabilidad	<i>CloudWatch</i>	<i>Logging</i> granular y detección de anomalías en tiempo real

Cuadro 4.1: Estructura por capas de la arquitectura *serverless* propuesta.

4.1.2. Convención de respuestas *HTTP*

Las funciones *Lambda* devuelven *HTTP* 200 cuando el resultado de negocio es exitoso y si hay un fallo en el procesamiento de datos o de conexión a internet siempre se devolverá un código de error 400/500. Por otro lado los códigos que se han procesado correctamente pero cuyo resultado no es favorable contendrán la información en el mensaje.

- 200 OK + message: OK → Éxito de negocio
- HTTP + message: BANNED/UPDATE_REQUIRED/EXPIRED... → Procesado y denegado
- 400/500 ERROR → Fallo técnico del servidor

Estos son algunos de los ejemplos que puede devolver *Lambda*, dependiendo del caso y de la función llamada. Son solo ejemplos que sirven para ilustrar de una forma más precisa los flujos de las distintas funciones *Lambda* detalladas en este capítulo.

Esta convención permite que *Unity* use un único punto de entrada para procesar todas las respuestas.

La Figura 4.2 muestra el flujo completo de comunicación entre el cliente *Unity* y el *backend serverless* desplegado en AWS. A la izquierda, *Unity* envía peticiones JSON a través de *API Gateway*, que actúa como punto de entrada y primera capa de validación. Desde ahí, cada petición es dirigida a su función *Lambda* correspondiente: *CheckDemoStatus* (que consulta la tabla *DemoAccess*), *RegisterSession* (que persiste datos en *PlayerStats*), *VerifyPlayerStats* (que valida métricas con *PlayerStats* si encuentra un *nonce* válido en *SessionNonces*) y *VerifyGameHash* (que contrasta el *hash* del ejecutable con la tabla *HistoricExeHashes*). Las respuestas regresan al cliente de nuevo, habiendo pasado por *API Gateway*, representado a la derecha. Cada *Lambda* opera de forma independiente y sin estado, accediendo únicamente a las tablas de *DynamoDB* que le corresponden según el principio de mínimo privilegio.

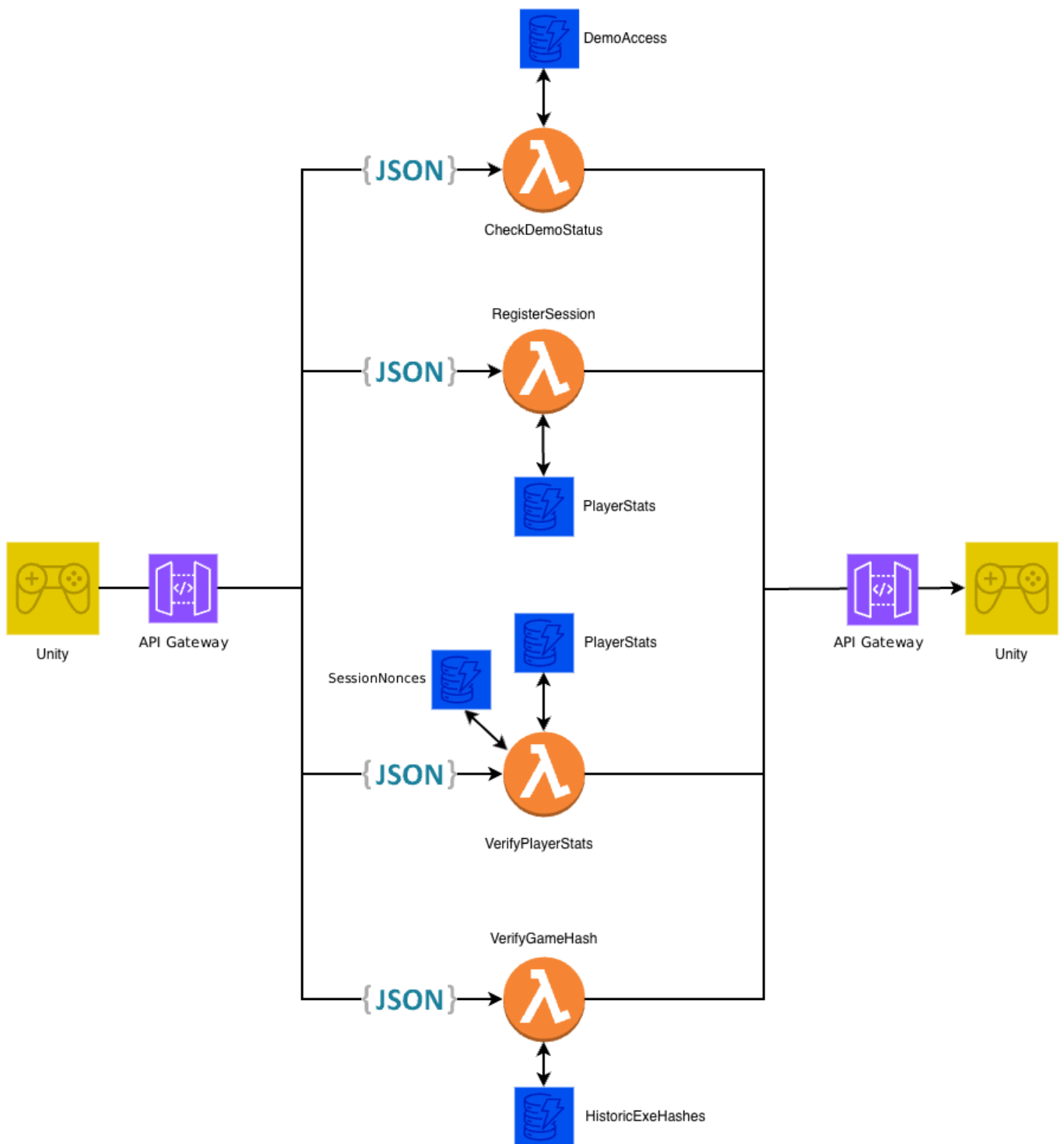


Figura 4.2: Vista general de la arquitectura *backend* propuesta en AWS

Cuando un usuario accede al portal web e inicia sesión, *Cognito* verifica sus credenciales y le entrega los *tokens JWT* que lo identifican. Con ese *token*, el portal solicita a *API Gateway* el enlace de descarga; *Gateway* comprueba que el *token* es válido y pasa la petición a *Lambda*, que genera un enlace prefirmado de *S3* con una ventana de expiración corta. El usuario recibe ese enlace directamente en el navegador y descarga el ejecutable desde *S3*, sin que el *bucket* quede expuesto en ningún momento ni haya que repartir credenciales entre los clientes. El diagrama de flujo se muestra en la Figura 4.3.

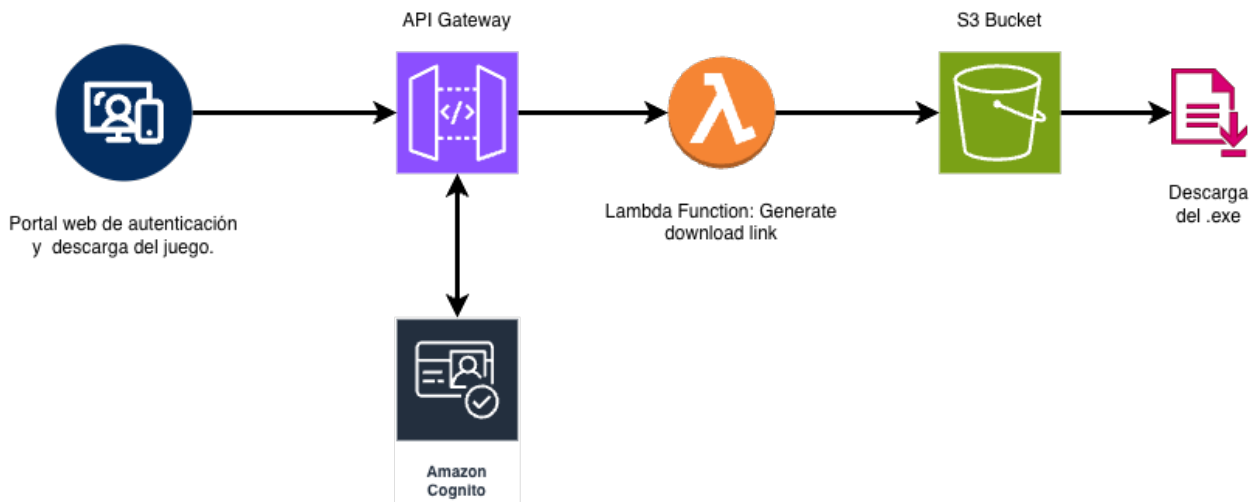


Figura 4.3: Flujo de autenticación y descarga desde el portal *web*

4.2. Descripción de los flujos principales

4.2.1. Flujo de VerifyPlayerStats: *Unity* → *AWS* → *Unity*

El proceso de verificación sigue un patrón cliente-servidor *request-response* optimizado para baja latencia. Cuando se arranca sesión, el cliente solicita al servidor un *nonce* efímero mediante una petición autenticada con el *token Cognito* del usuario. El servidor genera un *UUID*¹ único junto con un *timestamp* de 60 segundos y se lo manda al cliente. El cliente verifica el *nonce*, la telemetría del jugador (puntuación, tiempo jugado) y el *token* de sesión activo, y lo envía a *Lambda*.

Lambda valida el *nonce* de forma atómica mediante una *ConditionExpression* que comprueba simultáneamente que ese *nonce* pertenece al usuario autenticado, que no ha sido usado previamente y que no ha caducado. Si una o varias de estas condiciones fallan, la solicitud se rechaza. Este diseño traslada la autenticación íntegramente al servidor: el cliente no posee ningún secreto compartido, por lo que la interceptación del *nonce* no le da al atacante ninguna garantía ni posibilidad de manipular los datos.

Si el cliente reporta métricas inválidas o manipuladas, *Lambda* detecta la anomalía, restablece las estadísticas a cero y activa la bandera de “tramposo” en *DynamoDB*, devolviendo el código *CHEAT_WARNING* a *Unity*. Si el jugador es reincidente, se ejecuta el protocolo de revocación de usuario, mediante *admin_disable_user* se le inhabilita la

¹Universally Unique Identifier, valor único utilizado para identificar de forma inequívoca un elemento dentro de un sistema.

cuenta y se cierra la sesión global con `admin_user_global_sign_out`, negando el acceso a su cuenta con el mismo usuario/correo, impidiendo así también la creación de futuras cuentas con las mismas credenciales.

Si el *nonce* es válido se procede a validar las métricas enviadas por el usuario contra los datos ya almacenados en *DynamoDB*. Se rechazan las métricas desactualizadas, priorizando las que ya están almacenadas en el servidor. Si las métricas coinciden con las del servidor o representan una diferencia ínfima, se dan por válidas. *Lambda* confirma a *Unity* permitiendo continuar con el *gameplay* sin intervención adicional.

Si la firma esperada no es la correcta o se detecta manipulación local, restablece las estadísticas a cero, marca la bandera de tramposo en *DynamoDB* y envía a *Unity* el código 200 con la acción BANNED. Si el usuario vuelve a hacer trampas y se verifica que la bandera de tramposo está ya activada en *DynamoDB*, se activará el protocolo de revocación de *token* de usuario bloqueando así futuros accesos o registros con dicho correo o usuario.

Para usuarios sin registro previo en *DynamoDB*, *Lambda* responde NEW_USER y el flujo continúa con la carga de datos, que al no tener registros almacenados en la base de datos, se crean con valores iniciales de Score=0 y TimePlayed=0.

Nota: Este enfoque de inicialización asistida por cliente preserva *Zero Trust* porque:

- *Lambda* mantiene autoridad absoluta de lectura/validación.
- *Unity* solo crea registros triviales (valores cero).
- Todas las actualizaciones posteriores requieren validación en el *backend*.

La Figura 4.4 muestra el proceso completo de validación de estadísticas. Esta muestra las decisiones que toma *Lambda* en función de la validez del *nonce*, el estado de la sesión y la coherencia lógica de los datos, así como los códigos de respuesta que se devuelven a *Unity* en cada caso. El estado FORCE_CLOUD, hace referencia a la existencia de una ligera discrepancia de datos entre los locales del cliente y los almacenados en *DynamoDB*, sin ser necesaria una manipulación, solo una pequeña desincronización. En este caso, *Lambda* instruye a *Unity* para que se descarte el estado local y se adopten los datos del servidor sin penalizar al jugador.

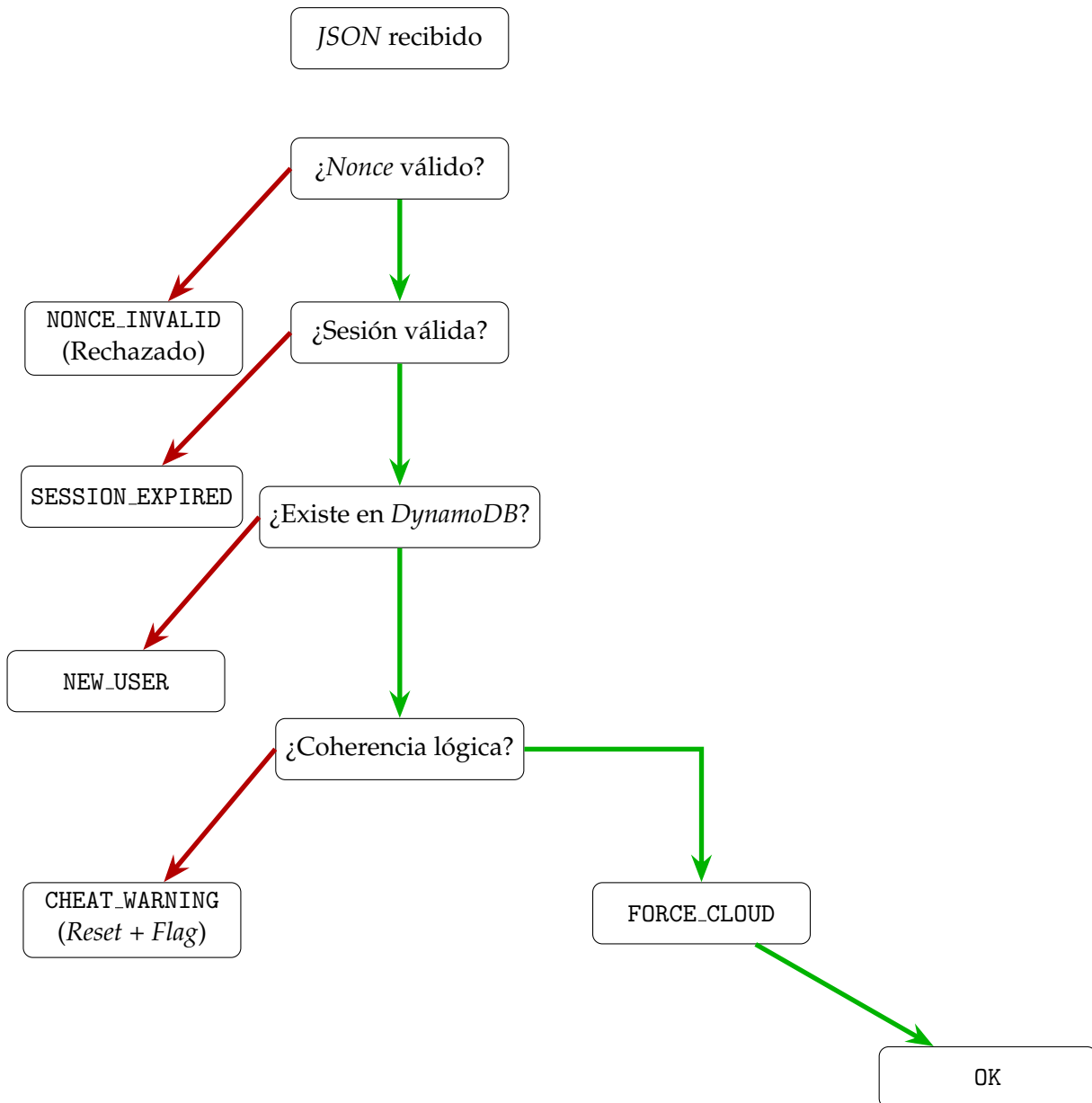


Figura 4.4: Proceso de validación de estadísticas del jugador y resolución de inconsistencias

4.2.2. Flujo de VerifyGameHash: *Unity* → *AWS* → *Unity*

La comprobación de si el código *hash* del ejecutable es válido se realiza en la función *Lambda VerifyGameHash*. El cliente envía un *JSON* cuyo contenido es el código *hash*, este lo recibe la función y verifica si se encuentra en la tabla en la que se almacenan los códigos válidos.

Al recibir el código se contrasta con los de la tabla de *DynamoDB* siendo posibles tres escenarios diferentes.

- Si el *hash* no se encuentra entre los códigos de la tabla, lo cual significa que se ha modificado el ejecutable, se envía el código de error *DENIED*.

- En caso de encontrarse el código, pero sin la bandera que indica que es el último introducido activada, se envía el código de actualización necesaria. Esto se hace ya que al encontrarse en la tabla se trata de un *hash* que en alguna versión anterior se introdujo y por lo tanto no se ha hecho ninguna manipulación del ejecutable.
- En el último de los casos, la función ha recibido un valor que se encuentra en la tabla y cuya bandera está activa devolviendo OK, permitiendo así la sesión en ese ejecutable.

La Figura 4.5 ilustra cómo *Lambda* decide la respuesta al recibir el *hash* del ejecutable. La función contrasta el valor entrante con la tabla `HistoricExeHashes` y se enfrenta a tres escenarios: el *hash* no aparece en la tabla, en cuyo caso se devuelve DENIED por considerar que el ejecutable ha sido modificado; el *hash* existe pero no tiene activa la bandera de versión vigente, lo que se resuelve con UPDATE_REQUIRED para forzar una actualización; o el *hash* coincide con la versión activa, momento en el que se devuelve OK y se permite continuar la sesión. Esta lógica permite mantener un historial de versiones legítimas y distinguir entre clientes desactualizados y clientes manipulados.

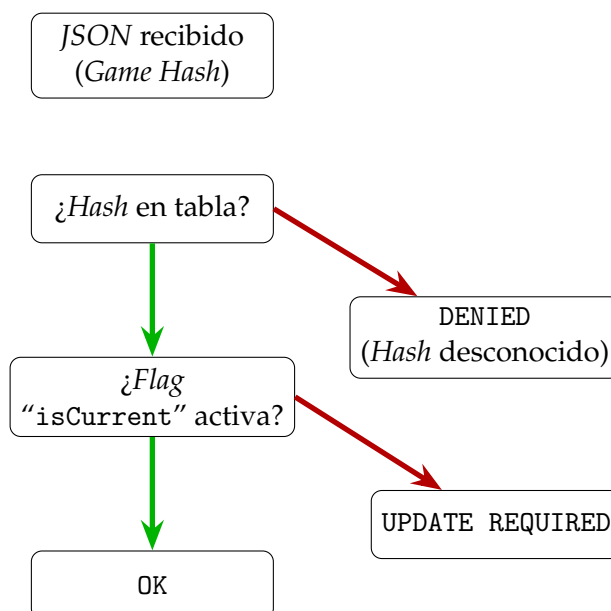


Figura 4.5: Proceso de verificación de integridad del ejecutable y gestión de versiones válidas.

4.2.3. Flujo de RegisterSession: *Unity* → AWS → *Unity*

`RegisterSession` aplica un sistema de exclusividad de sesión (anti multi-dispositivo), similar al que usan plataformas como *Netflix*, *HBO*, *Apple TV* o *Spotify*: cada usuario solo puede tener una sesión activa a la vez. Cuando el jugador entra o sale del juego, *Unity* envía a AWS un JSON con un *token* de sesión (un identificador único para esa partida) y una acción que indica si quiere registrarse ("register") o liberarse ("unregister"). A partir de ahí, *Lambda* obtiene el identificador del usuario desde el *token* de *Cognito* y utiliza *DynamoDB* como un mecanismo de exclusión de sesión: el campo que controla la sesión guarda el *token* de la sesión activa, o queda vacío cuando el jugador haya dejado de jugar.

Caso Logout: Libera la cuenta vaciando el campo de control de sesión, permitiendo futuros inicios de sesión. *Unity* ejecuta esto al cerrar la aplicación.

Caso Login:

1. Verifica que se incluya el *token* de sesión; si falta, responde 400 con ERROR: "Falta el *token*".
2. Consulta los datos del usuario en *DynamoDB*.
3. Si ya existe una sesión activa: bloquea el acceso con código 200, niega el acceso y registra *log* de intruso.
4. Si la cuenta está libre: asigna exclusividad guardando el nuevo *token* de sesión, registra *log* de acceso concedido y responde 200 con "OK Sesión registrada".

La Figura 4.6 muestra cómo *Lambda* procesa una petición de RegisterSession en función del campo *action* y del estado de la sesión en *DynamoDB*. Si el cliente envía *unregister*, el flujo libera el campo de sesión y termina con OK. Si la acción es *register*, se comprueba primero que el *sessionToken* esté presente (en su ausencia se devuelve ERROR 400) y a continuación se consulta el estado en *DynamoDB*: si ya existe una sesión activa para esa cuenta, se rechaza con SESSION IN USE y se registra el intento; si la cuenta está libre, se asigna el nuevo *token* mediante un *UpdateItem* condicional y se confirma la operación con OK. Esta condicionalidad garantiza la exclusividad atómica de la sesión incluso ante peticiones simultáneas desde varios dispositivos.

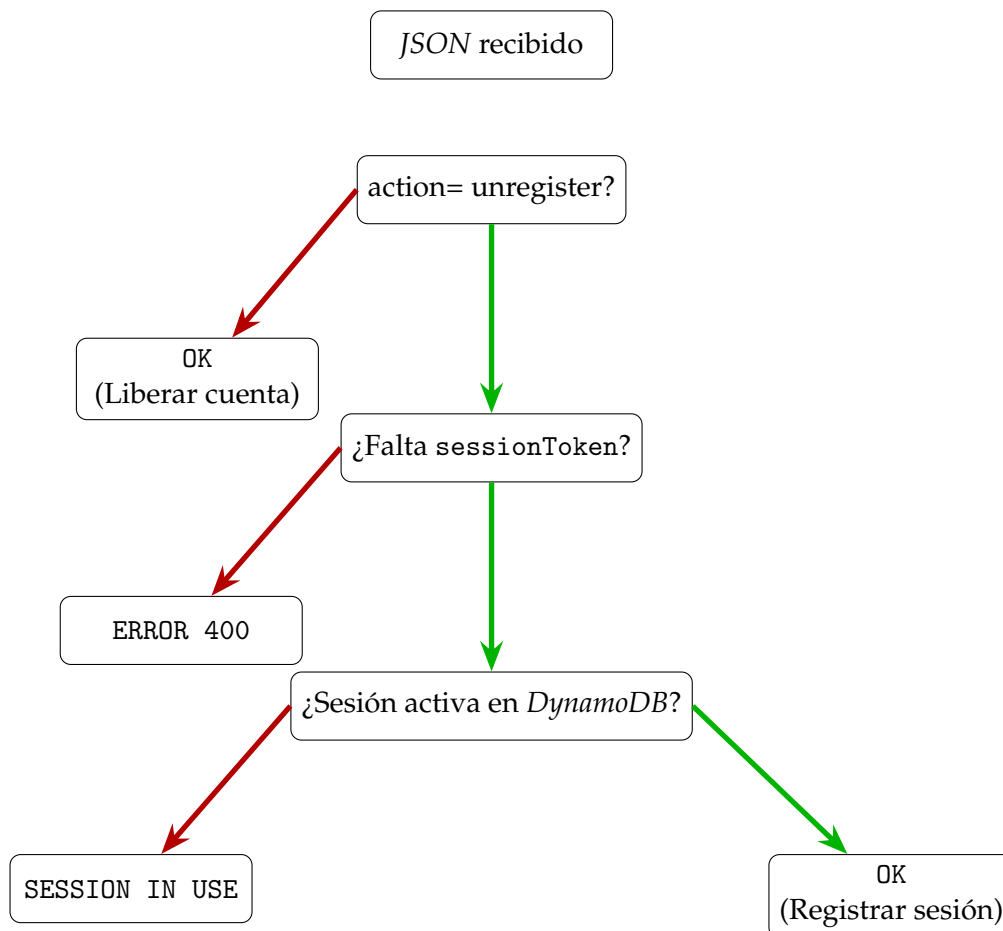


Figura 4.6: Flujo de RegisterSession

4.2.4. Flujo de CheckDemoStatus: *Unity* → *AWS* → *Unity*

La función `CheckDemoStatus` se implementa para gestionar a los usuarios un acceso temporal al producto, en este caso un videojuego. Tras la primera vez que inicia sesión o se registra el jugador este tendrá un tiempo limitado en el que disponga del producto.

Después de acceder al juego por primera vez con una cuenta nueva la función creará, para ese usuario con un *ID* único, unos parámetros en la tabla de *DynamoDB* asociada con la función *Lambda*. Estos parámetros son regulables desde la función o desde la propia tabla. Entre los datos se encuentran el primer y último instante en el que se inició sesión y el último instante permitido, esto se hace, mediante marcas temporales Unix (*epoch*), es decir, segundos transcurridos desde el 1 de enero de 1970 en *UTC*². También hay variables de tiempo total de juego permitido y tiempo usado, permitiendo así la gestión de tiempo disponible por fecha o por tiempo jugado.

Con el usuario ya registrado en la tabla, este dispone de un tiempo de juego, de la posibilidad de jugar hasta una fecha concreta o de ambas. La función devuelve `OK` si siguen cumpliéndose las condiciones para que la sesión siga disponible, `EXPIRED` si se intenta entrar tras la caducidad de la fecha o `TIME_LIMIT_REACHED` si se cumplen las horas disponibles de la *demo*.

La Figura 4.7 muestra el árbol de decisiones que sigue *Lambda* al recibir una petición de `CheckDemoStatus`. El flujo comienza comprobando si el usuario ya tiene un registro en *DynamoDB*; si no existe, se crea con los parámetros iniciales (instantes de primer y último acceso, fecha límite y tiempo de juego permitido) antes de continuar la evaluación. A partir de ahí, la función comprueba dos condiciones de caducidad: si la fecha límite se ha superado, devuelve `EXPIRED`; si el tiempo total de juego se ha agotado, devuelve `TIME_LIMIT_REACHED`; y, en caso contrario, `OK`, autorizando que la sesión continúe. Esta separación entre fecha y tiempo acumulado permite gestionar accesos temporales tanto por ventanas de calendario como por horas jugadas, o por una combinación de ambas.

²Coordinated Universal Time, escala horaria de referencia mundial respecto a la cual se definen el resto de zonas horarias.

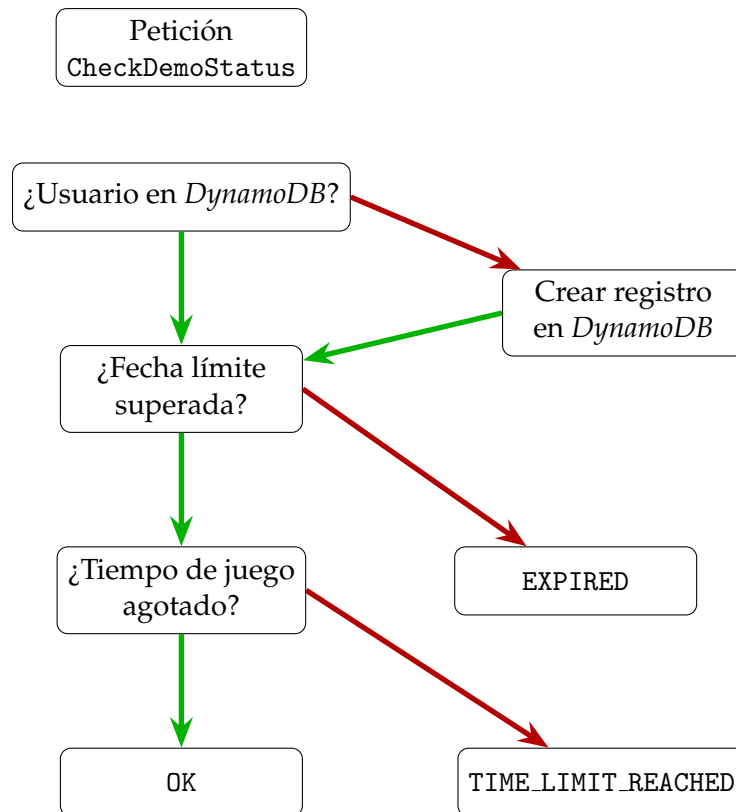


Figura 4.7: Flujo de control de acceso temporal en `CheckDemoStatus`

4.2.5. Registro de usuarios

Las funciones y la página web utilizan el mismo *user pool* de *Cognito*. Todas las funciones de *Lambda* se conectan con *Unity* a través de *API Gateway*. Estas *APIs* usan como autorizador a la *pool* ya mencionada.

La *web* cuenta con la opción de registrarse para crear una nueva cuenta, en este apartado se pide un nombre de usuario, correo electrónico y contraseña para la creación de la cuenta. Una vez se obtienen unas credenciales válidas, ni el usuario ni el correo pueden existir en la base de usuarios y la contraseña debe contar con unos requisitos y caracteres mínimos, se mandará un código de verificación al correo.

Una vez activada la cuenta con la verificación del correo, esta será la cuenta con la que se identifique el usuario y con la que iniciará sesión en el juego. De esta forma, todas las funciones tienen acceso a los usuarios que se encuentran jugando, permitiendo tomar medidas con cada cuenta de forma individual.

4.2.6. Flujo de `GenerateDownloadLink`: *Frontend* → *AWS* → *S3* → *Frontend*

`GenerateDownloadLink` crea enlaces temporales seguros para descargar el juego o cualquier otro archivo. Su uso queda a disposición del desarrollador. Tras iniciar sesión en la web del *launcher*, el *frontend* solicita el enlace enviando a *AWS* el *token* de sesión. *Lambda* accede a *Cognito* para verificar la autenticación para darle acceso al *bucket* de *S3*.

La función *Lambda* ya tiene designado el *bucket* desde el que debe servirse el archivo al usuario, accede al *bucket* y crea un enlace temporal válido por 5 minutos (campo serializado). Devuelve el enlace directo al navegador para que comience la descarga de inmediato. Si *Lambda* no localiza ningún archivo es que no se ha subido ningún archivo al *bucket* designado o que el tipo de archivo no es el designado.

Seguridad: Los enlaces solo se generan para usuarios autenticados; accesos sin permiso quedan bloqueados.

Casos:

1. Almacenamiento con paquete: Localiza el archivo, crea enlace temporal y confirma la descarga.
2. Almacenamiento sin paquete: Informa que no hay archivos preparados.
3. Sin autenticación: Se bloquea antes de procesar la petición.

La Figura 4.8 representa el flujo de `GenerateDownloadLink` desde el portal web hasta el *bucket* de S3. La solicitud entra con el *token* de sesión del usuario y la función comienza verificando que esté autenticado; si no lo está, se deniega el acceso antes de tocar el almacenamiento. Una vez confirmada la identidad, *Lambda* accede al *bucket* designado y comprueba si existe el archivo solicitado: si no encuentra ninguno, responde indicando que no hay paquete disponible; si lo localiza, genera una *URL* prefirmada con una ventana de validez corta y la devuelve directamente al navegador para iniciar la descarga. Este patrón evita repartir credenciales del *bucket* entre los clientes y limita la exposición del archivo al periodo de validez del enlace.

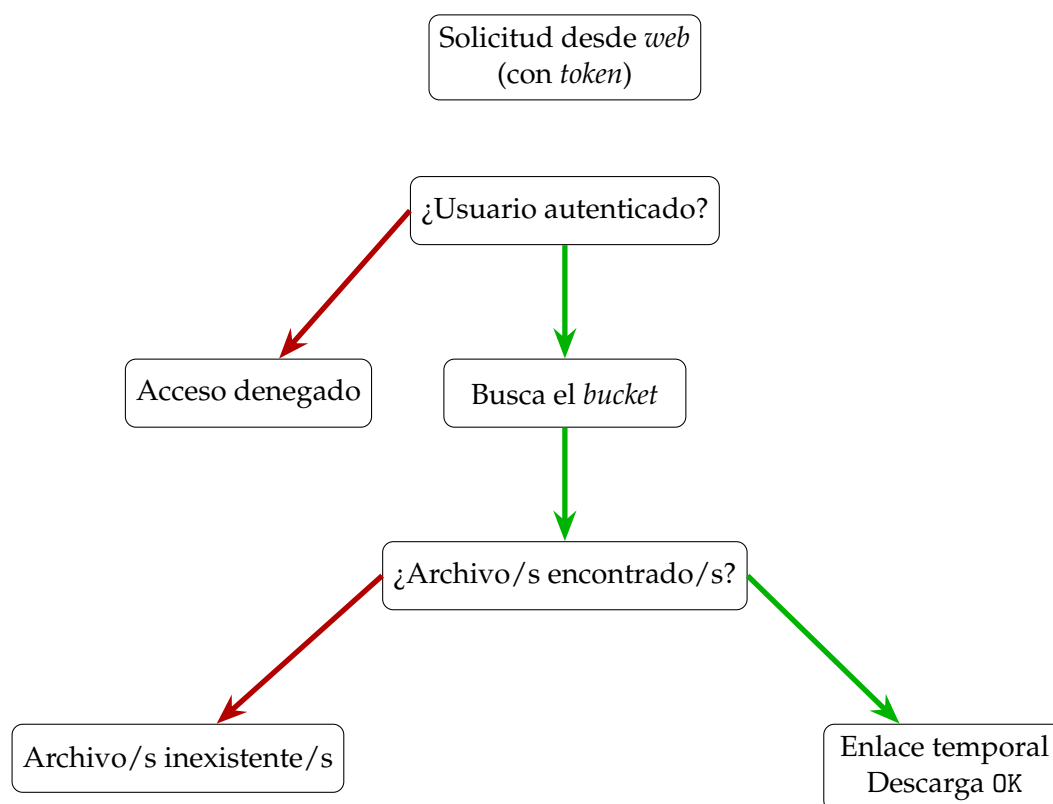


Figura 4.8: Flujo de generación de enlace temporal de descarga para usuarios autenticados.

4.3. Conclusiones del capítulo

La arquitectura propuesta centraliza la validación en *backend serverless*, desacoplando toda lógica crítica del cliente *Unity*. Los flujos implementados (verificación *HMAC*, sesiones exclusivas, *hashes* de ejecutables y descargas temporales) trasladan la autoridad al servidor mediante principios *Zero Trust*. Esta arquitectura contrarresta las posibles vulnerabilidades analizadas en el capítulo 2, cabe destacar que *Unity*, como motor orientado a la accesibilidad para desarrolladores, presenta una superficie de ataque más amplia que otros entornos más restrictivos. La relativa facilidad con la que se puede aplicar ingeniería inversa hace que depositar lógica crítica en el cliente suponga un riesgo [17], reforzando la idea de centralizar la validación en el *backend*. Esta arquitectura también permite trazabilidad completa gracias a *CloudWatch* para futuras auditorías si fuese necesario. Facilita el análisis práctico del Capítulo 5 proporcionando un modelo replicable para proyectos seguros de todo tipo.

Capítulo 5. Casos de Uso y Observación

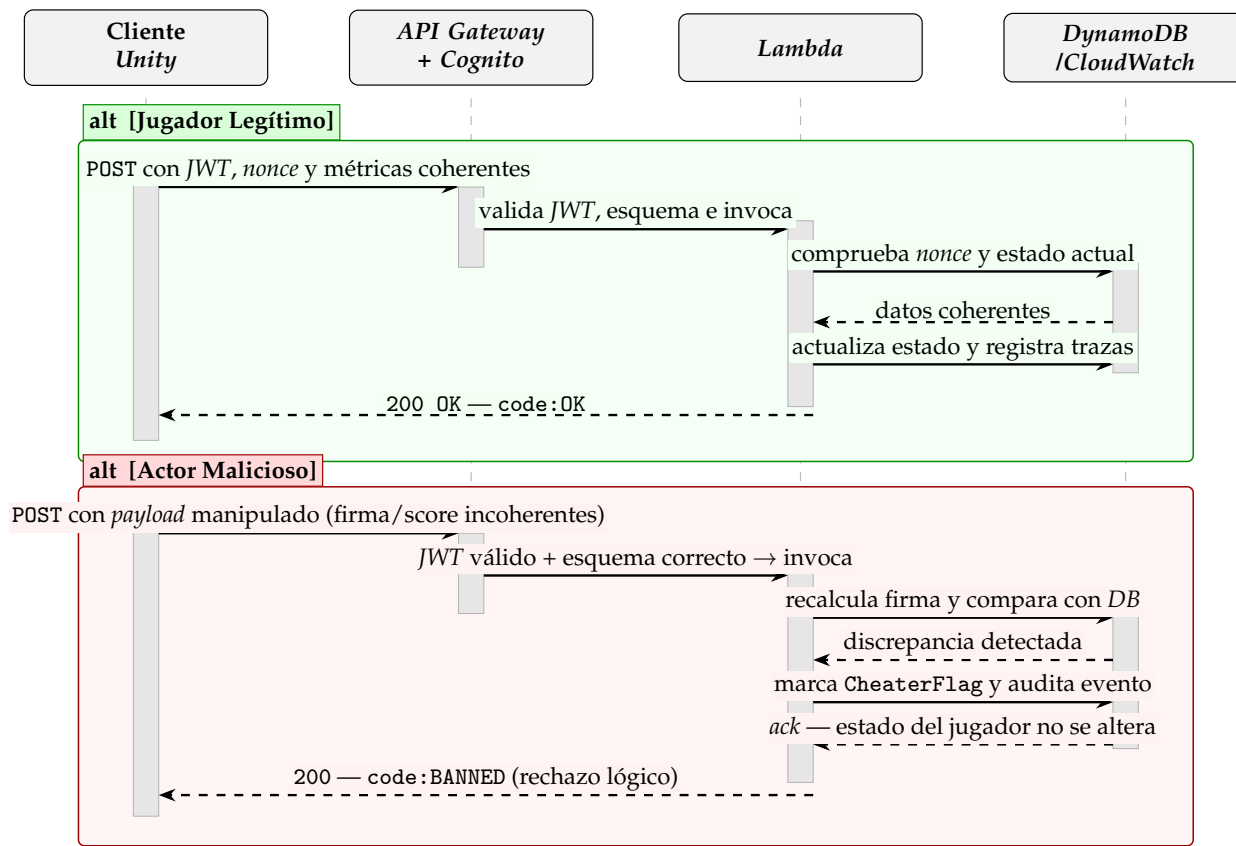


Figura 5.1: Diagrama de secuencia UML: jugador legítimo vs actor malicioso. Flujos de validación y rechazo transaccional en el *backend*.

5.1. Introducción y objetivo del capítulo

En este capítulo se presenta la observación sobre la arquitectura del proyecto mediante una serie de escenarios representativos. La Figura 5.1 adelanta el contraste entre un flujo legítimo y un intento malicioso. El objetivo es mostrar el comportamiento del sistema frente a situaciones de uso normales y adversas. De esta forma, se contempla la centralización de la lógica crítica en el *backend serverless*.

La revisión se divide en cuatro situaciones significativas para el proyecto: el comportamiento de un jugador legítimo, los intentos de manipulación de datos (*tampering*), la gestión del modo temporal o demo y los casos de desincronización entre el cliente y el servidor. Con estos escenarios propuestos, se puede comprobar si las decisiones de diseño tomadas previamente cumplen su función en la práctica, teniendo en cuenta la integridad de los datos, la trazabilidad de las acciones y las respuestas que da el sistema frente a eventos anómalos.

En todos los casos descritos, la observación se apoya en las rutas de *API Gateway*, *Cognito*, *Lambda* y *DynamoDB*, así como en los registros generados por *CloudWatch*. De esta

manera, el contenido del capítulo muestra el funcionamiento del sistema, y junto con el anexo 57 se documentan las evidencias con las que se puede observar el comportamiento y verificar que el cliente cuenta con un papel secundario, ya que la autoridad sobre datos sensibles como licencias, sesiones o puntuaciones reside en el servidor.

5.2. Metodología de pruebas y observación

La metodología de este capítulo se basa en la ejecución de varios escenarios representativos sobre el *backend* desplegado en *AWS*. Todos los casos se ejecutan sobre el mismo entorno, de esta forma, se garantiza que las observaciones sean reproducibles y que cualquier iteración posterior sobre la arquitectura sea replicable bajo las mismas condiciones.

Para los casos de uso se generan peticiones reales cubriendo tanto interacciones legítimas como intentos de manipulación de datos. Los datos generados con estas peticiones se observan con métricas y *logs* centralizados en *CloudWatch* y consultas sobre las tablas de *DynamoDB* para verificar la coherencia de datos tras cada escenario. La aparición de eventos marcados como *WARN* o la aparición de banderas que marcan a los usuarios como sospechosos, son algunas de las formas por las cuales se pueden detectar comportamientos fraudulentos.

5.3. Caso de uso 1: jugador legítimo

En este primer caso se analiza el comportamiento del sistema ante un jugador legítimo. Se explica el flujo completo desde el registro hasta la finalización de una partida sin realizar ninguna acción de manipulación.

El jugador inicia sesión, o se registra y verifica su correo, en la web, descarga el juego y accede a este mediante las credenciales gestionadas por *Cognito*, obteniendo *tokens JWT* válidos para las siguientes llamadas al *backend*. Durante la partida, *Unity* solicita un *nonce* efímero que posteriormente enviará a *Lambda* junto a los datos y eventos de telemetría de la sesión. Estas peticiones pasan por *API Gateway*, que valida la firma, la vigencia y el emisor del *token* además del formato del *JSON*. Por último, las funciones *Lambda*, una vez se ha validado el *nonce*, registran las puntuaciones y actualizan los perfiles almacenando la telemetría en *DynamoDB*.

Los registros de *CloudWatch* muestran un tráfico normal con peticiones correctamente formadas y sin ningún error o advertencia, como se recoge en la Figura 5.2. Las tablas de *DynamoDB* muestran actualizaciones correctas vinculadas a los usuarios sin ninguna bandera de alerta, tal como se observa en la Figura 5.3, y *Lambda* devuelve códigos *HTTP* 200 en cada función.

Listado 5.1: Códigos y *bodies* devueltos por las funciones *Lambda* en un acceso legítimo

```
VerifyPlayerStats
  "statusCode": 200,
  "body": "{\"code\": \"OK\", \"message\": \"Sincronizado.\"}"

VerifyGameHash
  "statusCode": 200,
  "body": "{\"code\": \"OK\", \"message\": \"Version actual.\"}"

RegisterSession
  "statusCode": 200,
```

```
"body": "{\"code\": \"OK\", \"message\": \"Sesion registrada.\"}"
```

2026-05-01T11:07:19.802Z	START RequestId: 567e366d-069a-48f0-9ae9-9ce5ccb56cc9 Version: \$LATEST
2026-05-01T11:07:19.827Z	=== ITEM LEIDO: CheaterFlag=False PermanentBan=None ===
2026-05-01T11:07:19.867Z	=== SCORES: local=0 db=0 TIMES: local=0.00 db=0.00 ===
2026-05-01T11:07:19.907Z	END RequestId: 567e366d-069a-48f0-9ae9-9ce5ccb56cc9
2026-05-01T11:07:19.907Z	REPORT RequestId: 567e366d-069a-48f0-9ae9-9ce5ccb56cc9 Duration: 105.12 ms

Figura 5.2: *CloudWatch: Log events* de *VerifyPlayerStats*

Userid (String) ▲	ActiveSessionToken ▼	CheaterFlag ▼	IPAddr... ▼	LastUpdated ▼	Score ▼	TimePlayed
a	<empty>			2026-05-01T...	6	7.72

Figura 5.3: *DynamoDB*: estado de la tabla *PlayerStats* tras una partida legítima

Este escenario confirma que la arquitectura funciona según lo previsto en condiciones normales, además de mostrar que la autoridad sobre el estado del juego reside en *AWS*, no en el cliente. El servidor mantiene el control sobre los datos sensibles incluso si el usuario se comporta correctamente.

5.4. Caso de uso 2: manipulación (*Tampering*)

En este segundo caso de uso se evalúa cómo reacciona el sistema cuando el cliente intenta manipular datos críticos, ya sea alterando el ejecutable, modificando el *JSON* de la puntuación o enviando métricas incoherentes con el tiempo de juego real.

El flujo, en caso de manipulación de datos, comienza igual que en el caso legítimo: el jugador se autentica con *Cognito*, obtiene un *token JWT* válido y realiza una partida. La diferencia aparece en el envío de resultados. Cuando el cliente manipulado genera una petición hacia el *endpoint* de envío de puntuaciones que, aunque pasa la validación estructural de *JSON Schema* en *API Gateway*, contiene una firma incoherente, o bien valores de puntuación desproporcionados respecto al tiempo jugado que contradicen el historial registrado en *DynamoDB*. *Lambda* al recibir el cuerpo de la solicitud recalcula la firma y la compara con la proporcionada por el cliente; si no coinciden, o si las métricas violan las reglas lógicas de progresión, se considera que ha habido *tampering*.

La función *Lambda* no actualiza el estado normal del jugador, sino que registra el incidente en la tabla correspondiente de *DynamoDB* marcando una bandera de usuario sospechoso, reinicia o corrige las estadísticas afectadas y escribe un evento de auditoría detallado (con *UserID*, *IP*, *DeviceID* y tipo de anomalía) en los *logs* de *CloudWatch*. Desde el punto de vista de *Unity*, la función responde con un *HTTP 200* pero incluye en el cuerpo un código *BANNED*, de forma que el cliente pueda reaccionar (bloqueando el acceso, mostrando un mensaje o forzando el cierre de sesión) sin necesidad de distinguir códigos *HTTP* diferentes. En escenarios de reincidencia, cuando *Lambda* detecta que la bandera

de tramposo ya estaba activa para ese usuario, se procede a revocar los *tokens* asociados impidiendo futuros accesos desde esa cuenta.

Listado 5.2: *Logs de CloudWatch* en primera y segunda manipulación de datos por parte del usuario “abc”

```
START RequestId : abefa9bf ...
=== ITEM LEIDO : CheaterFlag = None PermanentBan = None ===
=== SCORES: local=100 db=0 | TIMES: local=0.00 db=0.00 ===
=== DIFERENCIA SOSPECHOSA: diff_score=100 diff_time=0.00 ===
=== CHEAT FLAG : abc | Motivo : Puntuacion manipulada ===
=== VERIFICACION DB : CheaterFlag = True ===
END RequestId : abefa9bf ... Duration : 104.92 ms

--REINCIDENCIA--

START RequestId: 47b43a4b ...
=== ITEM LEIDO: CheaterFlag=True PermanentBan=None ===
=== SCORES: local=100 db=0 | TIMES: local=0.00 db=0.00 ===
=== DIFERENCIA SOSPECHOSA: diff_score=100 diff_time=0.00 ===
Cognito disable OK: abc
Sign-out global OK: abc
=== BAN PERMANENTE: abc | Motivo: Puntuacion manipulada ===
END RequestId: 47b43a4b ... Duration: 572.79 ms
```

Las Figuras 5.4 y 5.5 muestran el efecto de esa manipulación en *DynamoDB* y en el estado de la cuenta dentro de *Cognito*.

Userid (String) ▾	ActiveSessionToken ▾	CheaterFlag ▾	IPAddr... ▾	LastUpdated ▾	Score ▾	TimePlayed
abc	<empty>	true		2026-04-30T...	0	0

Figura 5.4: *DynamoDB*: estado de la tabla PlayerStats tras manipulación de datos

User name	Email address	Email verified	Confirmation status	Status
abc		Yes	Confirmed	⊖ Disabled

Figura 5.5: *Cognito*: estado de la *userpool* tras reincidencia de manipulación de datos

En el caso de que fuera la firma del ejecutable la que no coincide con la que tiene guardada el *backend* se impide el inicio de sesión al jugador. Esta discrepancia se puede dar por dos casos, siendo el primero por no tener la última versión del ejecutable y el segundo por tener una versión modificada de la aplicación.

Los logs de *CloudWatch* mostrarían el código UPDATE_REQUIRED en caso de tratarse de un hash o DENIED, dependiendo del caso. Al nivel de guardado de *hashes* en *DynamoDB* se guardan manualmente estos códigos y se marcan o desmarcan como válidos, como se muestra en la Figura 5.6. Haciendo esta distinción, *Lambda* mandará un código de negocio u otro en base a esta variable.

clientHash (String)	isCurrent
e5693ea177cf9939a24f...	true
d37c3729afc1befbfe40...	false

Figura 5.6: *DynamoDB*: ejemplo de la tabla *HistoricExeHashes* con un código válido y otro deshabilitado

Listado 5.3: Ejemplos de *JSONs* que recibe *Lambda* y los códigos y *bodies* devueltos por la función *VerifyGameHash*

```
JSON con hash habilitado:
{"clientHash" : "e5693ea177cf9939a24f..."}

Respuesta de Lambda:
{"statusCode": 200,
 "body": "{\"code\": \"OK\", \"message\": \"Sesion registrada.\"}"

JSON con hash deshabilitado:
{"clientHash" : "d37c3729afc1befbfe40..."}

Respuesta de Lambda:
{"statusCode": 426,
 "body": "{\"code\": \"UPDATE_REQUIRED\", \"message\": \"Tu version no es la mas reciente, actualiza el juego.\"}"

JSON con hash no perteneciente a la tabla HistoricExeHashes:
{"clientHash" : "invalidHashExample"}

Respuesta de Lambda:
{"statusCode": 403,
 "body": "{\"code\": \"DENIED\", \"message\": \"Esta version no esta autorizada.\"}"
```

Este contexto valida que la capa de verificación en *Lambda* es efectiva. Es capaz de detectar el fraude y de evitar la persistencia de datos corruptos al no actualizar el estado normal del jugador, marcándolo en su lugar como usuario sospechoso. La respuesta técnica mediante códigos de negocio permite que el cliente reaccione adecuadamente sin que el *backend* dependa de la integridad del dispositivo del jugador, cumpliendo así con el modelo *Zero Trust* planteado. Asimismo, la capacidad de distinguir mediante el *hash* si una discrepancia se debe a una versión desactualizada o a una modificación maliciosa del ejecutable, permite aplicar respuestas específicas para cada caso. Esto refuerza el control de integridad del cliente, garantizando que solo las versiones autorizadas tengan acceso, sin trasladar la autoridad de decisión al entorno inseguro del usuario.

5.5. Caso de uso 3: *demo* o modo limitado

En este caso de uso se evalúa el comportamiento del sistema cuando un jugador accede al juego con una sesión de *demo*, es decir, con los mismos mecanismos de autenticación que un usuario normal, pero sujeto a límites de tiempo de uso definidos en el *backend*.

El flujo comienza cuando un jugador nuevo inicia sesión por primera vez. Tras autenticarse se crea un registro en la tabla *DemoAccess* de *DynamoDB*, que está asociada con la función *Lambda* *CheckDemoStatus*. Esta tabla cuenta con parámetros configurables como el primer y último instante de inicio de sesión, el último instante permitido, el tiempo total de juego disponible y el tiempo ya consumido. A partir de ese momento, cuando *Lambda* consulta y actualiza la tabla, se evalúa si el usuario permanece dentro de la ventana de acceso autorizada, comprobando tanto si se ha superado la fecha límite como la disponibilidad de tiempo de juego.

Durante la ejecución, los códigos devueltos por la función reflejan las transiciones de estado, mostrando las entradas exitosas con código OK o los rechazos mediante EXPIRED o TIME_LIMIT_REACHED. Además, en *DynamoDB* se pueden observar las actualizaciones de los datos de los jugadores, siempre que se acceda exitosamente, pudiendo ver tanto el tiempo jugado como la última fecha disponible, entre otras métricas. La Figura 5.7 recoge un ejemplo de ese estado.

userId (String) ▲	expirationEpoch ▼	firstLoginEpoch ▼	lastCheckEpoch ▼	playTimeSeconds ▼	usedSeconds
testUser1	1777804357	1777717718	1777718218	7200	293
testUser2	1777718345	1777716284	1777718325	7200	1510
testUser3	1777804367	1777709832	1777718362	7200	7503

Figura 5.7: *DynamoDB*: tabla *DemoAccess* a fecha epoch 1777718362

Listado 5.4: Códigos y *bodies* devueltos por *CheckDemoStatus* en accesos de los usuarios “testUser1”, “testUser2” y “testUser3” respectivamente

```
"statusCode": 200,
"body": "{ \"code\": \"OK\", \"message\": \"Acceso permitido.\", \"allowed\": true, \"usedSeconds\": 293, \"limitSeconds\": 7200 }"

"statusCode": 403,
"body": "{ \"code\": \"EXPIRED\", \"message\": \"Demo caducada.\" }"

"statusCode": 403,
"body": "{ \"code\": \"TIME_LIMIT_REACHED\", \"message\": \"Tiempo de demo agotado.\" }"
```

Con este caso de uso se confirma la robustez del sistema para aplicar restricciones temporales sin requerir cambios en el cliente, ya que es el *backend* el que centraliza la lógica de acceso. Al delegar esta comprobación exclusivamente en *Lambda* y *DynamoDB*, se garantiza que las limitaciones sean inalterables por el usuario final, manteniendo el control de la demo bajo una política de acceso centralizada que responde correctamente interpretando los códigos de estado devueltos por el servidor.

5.6. Caso de uso 4: desincronización cliente–servidor

En este caso de uso se analiza la desincronización entre el estado local del cliente y el estado mantenido por el servidor.

El escenario contempla situaciones comunes como una sesión caducada, una versión del juego obsoleta o datos de partida que no coinciden con los registros del *backend*. Cuando *Unity* intenta continuar una partida o enviar una acción, la función *Lambda* correspondiente recibe la petición y ejecuta las validaciones definidas, detectando que los datos no son válidos conforme a las políticas actuales del sistema.

Los registros de *CloudWatch* permiten reconstruir estas discrepancias, identificando qué usuario o sesión las origina y qué política de corrección aplica el servidor en cada evento. En los *logs* se pueden apreciar peticiones desactualizadas con respuestas de tipo *FORCE_CLOUD* o errores 4XX, lo cual facilita la depuración y auditoría de estos eventos mediante la observabilidad operativa. La Figura 5.8 muestra el estado de *DynamoDB* usado como referencia para forzar la sincronización con la nube.

UserId (String) ▾	ActiveSessionToken ▾	Score ▾	TimePlayed
<u>S</u>	abc123-session-token	200	100

Figura 5.8: *DynamoDB*: ejemplo del usuario “S” en la tabla *PlayerStats*

Listado 5.5: *JSON* desactualizado que recibe *Lambda* y el código y *body* devuelto por *VerifyPlayerStats*

```

Usuario y body del JSON recibido:
  "cognito:username": "S"
  "body": "{\"data\": \"{\\\"score\\\":0,\\\"timePlayedSeconds\\\":0}\\\",\\\"sessionToken\\\":\\\"abc123-session-token\\\",\\\"nonce\\\":\\\"550e8400-e29b-41d4-a716-446655440000\\\"}\"},
Respuesta devuelta por Lambda:
  "statusCode": 200,
  "body": "{\"code\": \"FORCE_CLOUD\", \"message\": \"Partida local antigua.\"}"

```

Este escenario permite representar situaciones típicas como una sesión caducada, una *demo* expirada, una versión del cliente que ya no es válida o unos datos de partida que han quedado obsoletos respecto al servidor. En todos los casos, el principio es que el servidor actúe como fuente de verdad y el cliente debe adaptarse al estado centralizado, evitando que se consoliden acciones sobre datos inconsistentes o desfasados.

5.7. Limitaciones y posibles optimizaciones

Aunque la arquitectura propuesta demuestra la viabilidad de trasladar la validación al servidor y reforzar la integridad de los datos críticos del videojuego, el proyecto presenta varias limitaciones derivadas de su alcance y de las decisiones de diseño adoptadas. La primera es que el trabajo se centra en la seguridad lógica y en la coherencia del estado, pero no aborda los problemas de redes multijugador. Esto encaja con el alcance del proyecto, que excluye la parte de sincronización de baja latencia y se orienta a licencias, progresión, puntuaciones y autenticación.

Otra limitación importante es que aunque el control de *hash* del binario detecta modificaciones del ejecutable, el cliente sigue siendo vulnerable a técnicas avanzadas

como inyección en memoria, *proxies* de red o ingeniería inversa que preservan el *hash* mientras alteran la lógica en *runtime*. La arquitectura mitiga estos riesgos mediante validaciones lógicas y telemetría, pero no elimina por completo la superficie de ataque del cliente.

También debe señalarse la dependencia de servicios gestionados de AWS como *API Gateway*, *Lambda*, *DynamoDB*, *Cognito* y *CloudWatch*. Esta elección simplifica el desarrollo, reduce la carga operativa y facilita la escalabilidad, pero introduce una dependencia tecnológica clara del proveedor y condiciona la portabilidad de la solución.

Desde el punto de vista experimental, la validación se realiza sobre un prototipo y sobre escenarios representativos, no sobre una carga masiva real de usuarios ni sobre un entorno de producción. Esto significa que los resultados obtenidos sirven para demostrar la viabilidad técnica del enfoque, pero no permiten extrapolar con precisión su comportamiento bajo picos intensos de tráfico, abuso sostenido o condiciones de uso extremas. En consecuencia, los resultados obtenidos son útiles como evidencia de funcionamiento, pero no sustituyen una campaña de pruebas de rendimiento a gran escala.

Por último, el sistema protege activos concretos como licencias, sesiones, puntuaciones, *hashes* y telemetría, pero no cubre otros vectores de riesgo habituales en la industria, como la protección a nivel *kernel*, la detección avanzada de *cheats* persistentes o la prevención completa de piratería. Estas cuestiones quedan fuera del alcance por decisión metodológica y porque requerirían soluciones más invasivas, menos ligeras y menos compatibles con el objetivo del proyecto, que es ofrecer una arquitectura reproducible y aplicable a contextos independientes o académicos.

5.8. Síntesis de resultados y límites de la validación

Las pruebas realizadas han permitido comprobar que la arquitectura *serverless* propuesta funciona de forma adecuada en el contexto planteado. Se ha verificado que la validación puede trasladarse del cliente a un entorno controlado en AWS, centralizando la lógica en funciones *Lambda* y registrando la telemetría en *DynamoDB*. Esto ayuda a proteger los datos críticos y a detectar comportamientos anómalos. Además, *CloudWatch* resulta útil para la observabilidad y la auditoría del sistema.

Sin embargo, la validación tiene algunas limitaciones. Las pruebas se han hecho en escenarios controlados, por lo que no permiten asegurar el rendimiento ante cargas masivas o ataques *DDoS*. Además, el cliente sigue siendo vulnerable a técnicas como la ingeniería inversa o la inyección en memoria. La propuesta se centra en la seguridad lógica y la coherencia del estado, sin abordar problemas propios de juegos multijugador masivos, como la latencia o la sincronización en tiempo real.

La Figura 5.9 recoge la latencia por función *Lambda* y el reparto de eventos detectados en *CloudWatch* durante los escenarios anteriores. Las cifras provienen únicamente de esas ejecuciones.

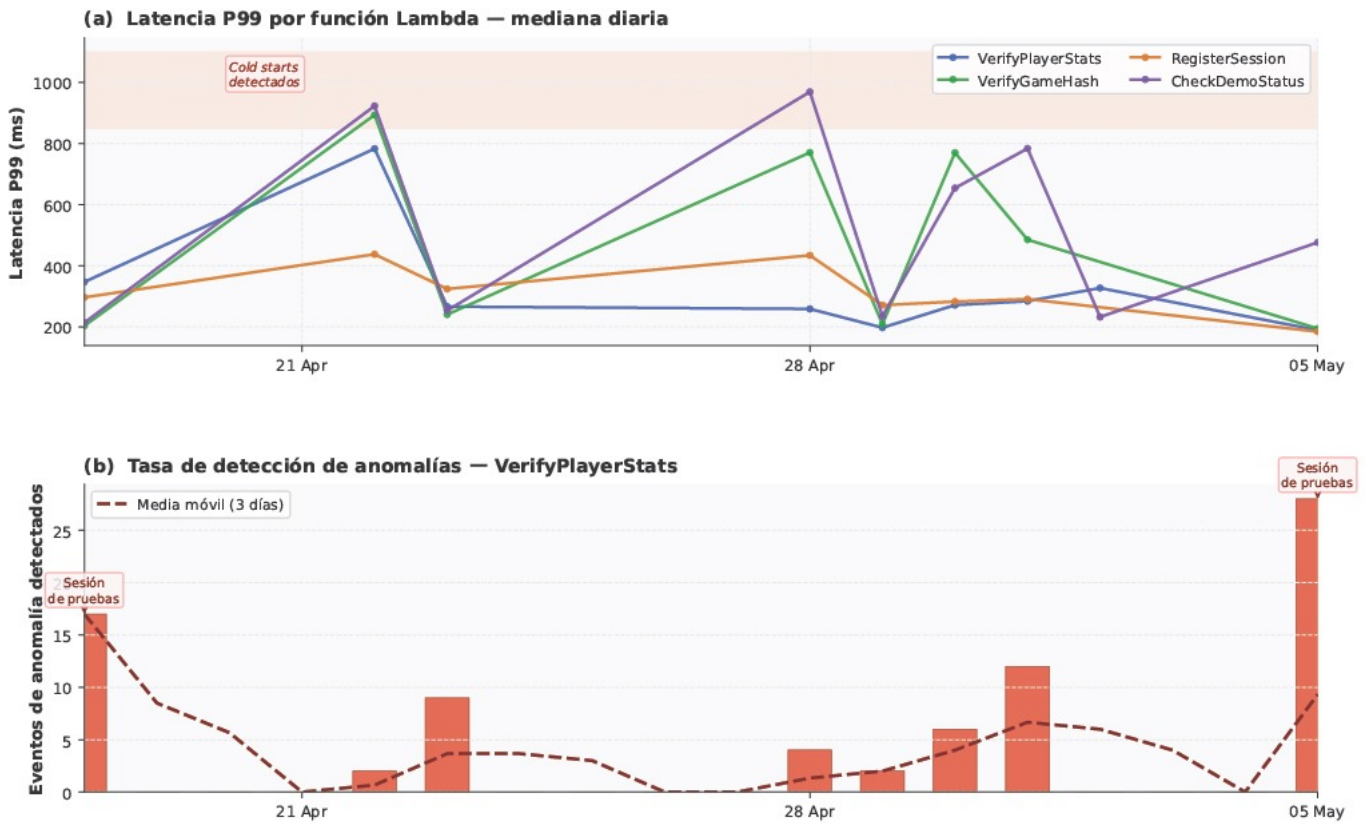


Figura 5.9: Latencia P99 por función *Lambda* (a) y tasa de detección de anomalías en *VerifyPlayerStats* (b). Datos exportados desde *Amazon CloudWatch*. Periodo: 18 abr – 5 may 2026.

Capítulo 6. Conclusiones y trabajo a futuro

Este trabajo ha cumplido el objetivo de poder diseñar una arquitectura *serverless* que traslade la autoridad de validación del cliente al servidor. Tras la implementación, el despliegue, diseño y la observación de la arquitectura sobre los casos de uso descritos en el Capítulo 5, se han extraído las siguientes conclusiones:

- **Validación del cambio de paradigma:** Tras poner a prueba y examinar los escenarios del jugador legítimo y de manipulación de datos (Casos de Uso 1 y 2) se ha mostrado que el cliente de *Unity* puede limitarse a una función de visualización, mientras que los datos clave y las decisiones sobre sesiones, registros, puntuaciones y licencias residen y son resueltos en *AWS*. Las trazas de *CloudWatch* que se han recogido, muestran que las puntuaciones persistidas anteriormente en *DynamoDB* provienen siempre de la validación en *Lambda*, y no del valor que reporta *Unity* (el jugador).
- **Sostenibilidad y escalabilidad:** Con el modelo *serverless* se ha logrado una arquitectura que brinda al desarrollador la oportunidad de que la arquitectura escale de forma automática según la demanda por parte de los jugadores, reduciendo así los costes, convirtiendo este modelo de arquitectura en un modelo óptimo para estudios independientes y proyectos académicos.
- **Observabilidad como componente central de la seguridad:** La combinación de *DynamoDB* y *CloudWatch*, ha permitido construir un registro auditable de las interacciones del sistema, no solo de las interacciones críticas. Las trazas recogidas en el Capítulo 5 y en los anexos muestran que la telemetría no solo facilita la depuración para los desarrolladores, sino que brinda al desarrollador y a la plataforma trazas verificables ante intentos de manipulación, habilitando respuestas automatizadas, previamente programadas en *Lambda*, como la activación de la bandera *CheaterFlag*, la invalidación de estadísticas manipuladas por el jugador o la revocación de *tokens* mediante `admin.disable_user` y `admin.user_global_sign_out` en casos de reincidencia.
- **Eficacia del modelo *Zero Trust*:** La combinación de *API Gateway* y *Cognito* ha rechazado, en los escenarios planteados, las peticiones no autenticadas, las que presentaban un *token* inválido y las que no cumplían con el *JSON* esperado, antes de alcanzar la lógica de negocio. Este comportamiento es coherente con la hipótesis que se ha formulado en la introducción y con los criterios de aceptación definidos en el Capítulo 3.

Trabajo a futuro

La arquitectura propuesta cubre los objetivos planteados en este Trabajo Fin de Grado, aunque deja abiertas varias líneas de investigación y extensión que permitirían consolidar y ampliar los resultados obtenidos. Estas líneas se han agrupado en torno a tres ejes: la mejora de las capacidades de detección, la caracterización cuantitativa del sistema y la consolidación del prototipo como un modelo replicable.

- **Detección proactiva mediante *Machine Learning*:** Transicionar de una observabilidad fiable, pero estática, a un sistema inteligente que utilice modelos de aprendizaje automático para identificar patrones de fraude complejos y generar mejores datos para luego auditarlos. Aprovechando el gran volumen de datos recolectados por la telemetría, la *IA* podría detectar anomalías que la lógica estática actual no puede.
- **Análisis del coste operativo:** Una línea futura sería estudiar el coste real del sistema cuando se usa durante un periodo prolongado, viendo qué servicios pesan más en la factura (*Lambda*, *DynamoDB* o *CloudWatch*) y proponiendo ajustes para reducirlo. Esto reforzaría la idea de que la arquitectura es asumible para estudios pequeños y proyectos académicos.
- **Ampliación a redes multijugador en tiempo real:** Otra línea futura sería abordar la seguridad en redes multijugador. Esto implicaría indagar e investigar cómo se podría mantener la autoridad del servidor en flujos que requieren una latencia baja, evaluando alternativas como *AWS GameLift*, para no comprometer la experiencia del jugador. La viabilidad de este servicio en producción está respaldada por su uso en títulos como *Apex Legends*, *Dead by Daylight* o *Mortal Kombat 1*, así como por su capacidad demostrada para escalar hasta 100 millones de jugadores concurrentes [18].
- **Creación de un *framework* replicable:** El objetivo sería pasar del prototipo actual a un modelo totalmente replicable que permita a equipos pequeños adoptar principios de *seguridad por diseño* de manera eficiente y con un coste operativo reducido.

6.1. Aportes Individuales

El reparto de responsabilidades en la fase de implementación se ha articulado en torno a las funciones *Lambda* desplegadas en *AWS*.

6.1.1. Dorje Khampa Herrezuelo Blasco

- **GenerateDownloadLink:** genera enlaces prefirmados temporales de *S3* a partir del *token* de *Cognito* del usuario, permitiendo descargar el ejecutable desde la web del *launcher* sin exponer credenciales del *bucket*.
- **RegisterSession:** garantiza la exclusividad de sesión por usuario mediante *DynamoDB*, gestionando los flujos de *login* y *logout* para impedir sesiones simultáneas en distintos dispositivos.

- **VerifyPlayerStats:** núcleo del sistema antifraude. Valida el *nonce* de cada partida, contrasta las métricas del cliente con *DynamoDB* y activa los protocolos de respuesta ante manipulación (*CheaterFlag*, reinicio de estadísticas y, en caso de reincidencia, revocación de credenciales).

6.1.2. Alberto Peñalba Martos

- Desarrollo de todo lo relacionado con *Cognito* y su integración con *Unity*, incluyendo la autenticación de usuarios, la gestión de *tokens JWT* y el enlace entre el cliente del juego y el sistema de identidad compartido con el portal web.
- **VerifyGameHash:** verifica la integridad del ejecutable contrastando su *hash* contra la tabla *HistoricExeHashes*, distinguiendo entre versiones manipuladas (*DENIED*), obsoletas (*UPDATE_REQUIRED*) y vigentes (*OK*).
- **CheckDemoStatus:** controla el acceso temporal al producto mediante ventana de fecha y tiempo de juego acumulado, respondiendo con *OK*, *EXPIRED* o *TIME_LIMIT_REACHED* según el estado del usuario.

6.1.3. Trabajo conjunto

De forma conjunta se han definido la arquitectura general del sistema, el modelo de datos en *DynamoDB* y la convención de respuestas *HTTP* compartida entre todas las funciones. El equipo ha desarrollado también el portal web del *launcher* y un prototipo de juego 2D en *Unity* que ha servido para validar la integración con *AWS* y verificar el comportamiento de las funciones en los casos de uso descritos en el Capítulo 5. La redacción de la memoria y la elaboración de las figuras se han llevado a cabo de forma colaborativa.

6.2. Repositorio

El código fuente de todas las funciones *Lambda* se puede encontrar en el siguiente enlace: <https://github.com/DorjeeHerBlas/TFG-25-26-AWS>

Chapter 6. Conclusions and future applications of the project

This work has fulfilled the objective of designing a serverless architecture that shifts validation authority from the client to the server. After implementing, deploying, designing, and observing the architecture across the use cases described in Chapter 5, the following conclusions have been drawn:

- **Validation of the paradigm shift:** After testing and examining the legitimate player and data tampering scenarios (Use Cases 1 and 2), it can be noted that the *Unity* client can be limited to a display function, while key data and decisions concerning sessions, records, scores, and licenses reside in and are resolved by *AWS*. The *CloudWatch* traces collected show that the scores previously persisted in *DynamoDB* always come from validation in *Lambda*, and not from the value reported by *Unity* (the player).
- **Sustainability and scalability:** With the *serverless* model, an architecture has been achieved that gives the developer the opportunity for the architecture to scale automatically according to player demand, thereby reducing costs. This makes this architectural model the appropriate one for independent studios and academic projects.
- **Observability as a core component of security:** The combination of *DynamoDB* and *CloudWatch* has made it possible to build an auditable record of system interactions, not only of critical interactions. The traces collected in Chapter 5 and in the appendices show that telemetry not only facilitates debugging for developers, but also provides the developer and the platform with verifiable traces in the face of tampering attempts, enabling automated responses, previously programmed in *Lambda*, such as the activation of the *CheaterFlag*, the invalidation of statistics manipulated by the player, or the revocation of *tokens* via `admin.disable_user` and `admin.user_global_sign_out` in cases of recidivism.
- **Effectiveness of the Zero Trust model:** The combination of *API Gateway* and *Cognito* has rejected, in the scenarios considered, unauthenticated requests, those presenting an invalid *token*, and those that did not comply with the expected *JSON*, before reaching the business logic. This behaviour is consistent with the hypothesis formulated in the introduction and with the acceptance criteria defined in Chapter 3.

Future work

The proposed architecture covers the objectives set out in this Final Degree Project, although it leaves open several lines of research and extension that would consolidate and broaden the results obtained. These lines have been grouped around three axes: improving detection capabilities, the quantitative characterisation of the system, and the consolidation of the prototype as a replicable model.

- **Proactive detection through *Machine Learning*:** Transitioning from reliable but static observability to an intelligent system that uses machine learning models to identify complex fraud patterns and generate better data for subsequent auditing. By taking advantage of the large volume of data collected through telemetry, *AI* could detect anomalies that the current static logic cannot.
- **Operational cost analysis:** A future line of work would be to study the real cost of the system when used over a prolonged period, identifying which services weigh most heavily on the bill (*Lambda*, *DynamoDB*, or *CloudWatch*) and proposing adjustments to reduce it. This would reinforce the idea that the architecture is affordable for small studios and academic projects.
- **Extension to real-time multiplayer networks:** A future line would be to address security in multiplayer networks. This would involve investigating how server authority could be maintained in flows requiring low latency, evaluating alternatives such as *AWS GameLift*, so as not to compromise the player's experience. The viability of this service in production is supported by its use in titles such as *Apex Legends*, *Dead by Daylight*, and *Mortal Kombat 1*, as well as by its demonstrated capacity to scale up to 100 million concurrent players [18].
- **Creation of a replicable *framework*:** The objective would be to move from the current prototype to a fully replicable model that allows small teams to adopt "security by design" principles efficiently and at a reduced operational cost.

6.3. Individual Contributions

The distribution of responsibilities during the implementation phase was organised around the *Lambda* functions deployed in *AWS*.

6.3.1. Dorje Khampa Herrezuelo Blasco

- **GenerateDownloadLink:** generates temporary pre-signed *S3* links from the user's *Cognito* token, allowing the executable to be downloaded from the *launcher* website without exposing the *bucket* credentials.
- **RegisterSession:** ensures session exclusivity per user through *DynamoDB*, managing the *login* and *logout* flows in order to prevent simultaneous sessions on different devices.
- **VerifyPlayerStats:** core of the anti-fraud system. It validates the *nonce* of each match, compares the client metrics with *DynamoDB*, and activates the response protocols when tampering is detected, such as the *CheaterFlag*, the reset of manipulated statistics and, in cases of repeated offences, the revocation of credentials.

6.3.2. Alberto Peñalba Martos

- Development of everything related to *Cognito* and its integration with *Unity*, including user authentication, *JWT token* management, and the connection between the game client and the identity system shared with the web portal.
- `VerifyGameHash`: verifies the integrity of the executable by comparing its *hash* against the `HistoricExeHashes` table, distinguishing between tampered versions (`DENIED`), outdated versions (`UPDATE.REQUIRED`), and valid versions (`OK`).
- `CheckDemoStatus`: controls temporary access to the product through a date window and accumulated playtime, returning `OK`, `EXPIRED`, or `TIME.LIMIT.REACHED` depending on the user's status.

6.3.3. Joint Work

The overall system architecture, the data model in *DynamoDB*, and the shared *HTTP* response convention used by all functions were defined jointly. The team also developed the *launcher* web portal and a 2D game prototype in *Unity*, which was used to validate the integration with *AWS* and verify the behaviour of the functions in the use cases described in Chapter 5. The writing of the report and the creation of the figures were also carried out collaboratively.

6.4. Repository

The source code of all the *Lambda* functions can be found at the following link: <https://github.com/DorjeeHerBlas/TFG-25-26-AWS>

Anexos

Anexo A: Función *Lambda* VerifyPlayerStats

Fase 1 — Arranque limpio

Primer arranque tras limpiar el registro. El *nonce* se emite y consume correctamente, y la verificación confirma que el jugador no tiene *flags* activos y el *score* local es coherente con el de la nube.

Listado 6.1: *CloudWatch* y *Lambda* — arranque limpio

```
START RequestId: efdcd4a4 ... Init Duration: 628.79 ms
START RequestId: 715f33fc ...
=== ITEM LEIDO: CheaterFlag=None PermanentBan=None ===
=== SCORES: local=2 db=0 | TIMES: local=4.60 db=5.00 ===
END RequestId: 715f33fc ... Duration: 129.40 ms

Respuesta devuelta por Lambda:
{
  "statusCode": 200,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Content-Type": "application/json"
  },
  "body": "{\"code\": \"OK\", \"message\": \"Sincronizado.\"}"
}
```

Fase 2 — Primera infracción (CHEAT_WARNING)

El usuario ha manipulado los datos, y la función VerifyPlayerStats lo ha detectado comparando las métricas con las ya almacenadas en *DynamoDB*. Se activa CheaterFlag, pero no se le revoca el acceso al no ser reincidente.

Listado 6.2: *CloudWatch* y *Lambda* — primera infracción

```
START RequestId: 12f78bdf ...
=== ITEM LEIDO: CheaterFlag=None PermanentBan=None ===
=== SCORES: local=18 db=7 | TIMES: local=22.43 db=19.55 ===
=== DIFERENCIA SOSPECHOSA: diff_score=11 diff_time=2.88 ===
=== CHEAT FLAG: d | Motivo: Puntuacion manipulada ===
=== VERIFICACION DB: CheaterFlag=True ===
END RequestId: 12f78bdf ... Duration: 104.92 ms

Respuesta devuelta por Lambda:
{
  "statusCode": 200,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Content-Type": "application/json"
  },
  "body": "{\"code\": \"CHEAT_WARNING\", \"message\": \"Aviso: Puntuacion manipulada. Stats reseteadas.\"}"
}
```

Fase 3 — Arranque con *flag* activo

El jugador ha iniciado sesión correctamente, enviando unas métricas sin manipular, pero CheaterFlag sigue activada. *Lambda* devuelve el mismo *output* que en la fase 1.

Listado 6.3: *CloudWatch* — arranque con *flag* activo

```
START RequestId: 2b77c88f ...
=== ITEM LEIDO: CheaterFlag=True PermanentBan=None ===
=== SCORES: local=0 db=0 | TIMES: local=24.54 db=0.00 ===
END RequestId: 2b77c88f ... Duration: 100.77 ms
```

Fase 4 — Reincidencia: *ban* permanente

El usuario vuelve a enviar unas métricas manipuladas. Al estar CheaterFlag activada se le revoca el acceso activando PermanentBan a true y *Lambda* procederá a deshabilitar el usuario.

Listado 6.4: *CloudWatch* y *Lambda* — *ban* permanente por reincidencia

```
START RequestId: 8633fb92 ...
=== ITEM LEIDO: CheaterFlag=True PermanentBan=None ===
=== SCORES: local=100 db=0 | TIMES: local=13.03 db=24.54 ===
=== DIFERENCIA SOSPECHOSA: diff_score=100 diff_time=-11.52 ===
Cognito disable OK: d
Sign-out global OK: d
=== BAN PERMANENTE: d | Motivo: Puntuacion manipulada ===
END RequestId: 8633fb92 ... Duration: 572.79 ms

Respuesta devuelta por Lambda:
{
  "statusCode": 403,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Content-Type": "application/json"
  },
  "body": "{\"code\": \"PERMANENT_BANNED\", \"message\": \"Baneo definitivo: Puntuacion manipulada\"}"
}
```

Tabla PlayerStats

La siguiente tabla es un ejemplo de los datos que podría contener PlayerStats. Esta es empleada tanto para consultar como para actualizar datos por las funciones *Lambda* VerifyPlayerStats y RegisterSession.

Las columnas contienen: el *username*, el *token* que indica si la cuenta está ahora mismo activa, una *flag* para marcar las cuentas que han manipulado sus datos, la dirección *IP*, el último registro, la puntuación y el tiempo de juego.

UserId	ActiveSessionToken	CheatFlag	IPAddress	LastUpdated	Score	Time
a	f221d0ea-ad3a...	false	XX.XX.XX	2026-04-29T...	3	3.12
b	<empty>	false	XX.XX.XX	2026-04-29T...	10	16.5
c	<empty>	true	XX.XX.XX	2026-04-29T...	9	14.6
d	<empty>	false	XX.XX.XX	2026-04-29T...	13	16.5
e	<empty>	false	XX.XX.XX	2026-04-29T...	71	103.9

Cuadro 6.1: Ejemplo de estado de tabla de *DynamoDB* PlayerStats

Tabla SessionNonces

El siguiente cuadro muestra cómo estaría la tabla SessionNonces con tres cuentas iniciadas en un periodo de tiempo muy breve. Esta tabla guarda los *nonces* efímeros que VerifyPlayerStats consulta que sea del usuario correspondiente y si no se ha usado (se marca si es así).

La tabla muestra el código del *nonce*, el tiempo de expiración o *TTL*, si se ha utilizado ya el *nonce*, y el *username*.

nonce	expiresAt	used	userId
1ba8e285-3e66...	1777540990	true	a
5df6a064-11a6...	1777540971	true	b
bddbb0ea-1004...	1777541023	true	c

Cuadro 6.2: Ejemplo de estado de tabla de *DynamoDB* SessionNonces

Anexo B: Función *Lambda* RegisterSession

Trazas de ejecución (*CloudWatch*)

Los *logs* recogen dos escenarios distintos: accesos legítimos en sesión normal y bloqueos por intento de acceso concurrente desde otro dispositivo.

Escenario 1 — Accesos legítimos consecutivos

Múltiples arranques del mismo usuario d en sesiones sucesivas. En cada caso el ActiveSessionToken anterior había sido liberado al cerrar el juego, por lo que la función *Lambda* concede el acceso y registra el nuevo *token* registrado nuevamente en la tabla PlayerStats de *DynamoDB*.

Listado 6.5: *CloudWatch* y *Lambda* — accesos legítimos consecutivos

```
START RequestId: 3ad09b93 ... Init Duration: 490.36 ms
Acceso concedido a d. Puerta bloqueada.
END RequestId: 3ad09b93 ... Duration: 435.26 ms

START RequestId: 794f6630 ...
Acceso concedido a d. Puerta bloqueada.
END RequestId: 794f6630 ... Duration: 67.47 ms

START RequestId: 6417f039 ...
Acceso concedido a d. Puerta bloqueada.
END RequestId: 6417f039 ... Duration: 80.24 ms
```

```

Respuesta devuelta por Lambda en los tres casos:
{
  "statusCode": 200,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Content-Type": "application/json"
  },
  "body": "{\"code\": \"OK\", \"message\": \"Sesion registrada.\"}"
}

```

Escenario 2 — Bloqueo por sesión en uso

El usuario *a* arranca el juego, mientras la sesión sigue activa, se intenta iniciar sesión en un segundo dispositivo. Cuando esto ocurre, *Lambda* detecta que `ActiveSessionToken` no está vacío y rechaza el acceso con `SESSION_IN_USE`. El primer dispositivo sigue jugando sin interrupción. Solo cuando el token se libera (cierre del juego en el primer dispositivo) el segundo acceso es concedido.

Listado 6.6: *CloudWatch y Lambda* — bloqueo por sesión en uso

```

START RequestId: 786352f2 ...
Acceso concedido a a. Puerta bloqueada.
END RequestId: 786352f2 ... Duration: 102.65 ms

START RequestId: 1922f9c6 ...
INTRUSO BLOQUEADO: a intento entrar desde otro PC.
END RequestId: 1922f9c6 ... Duration: 72.43 ms

Respuesta devuelta por Lambda:
{
  "statusCode": 200,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Content-Type": "application/json"
  },
  "body": "{\"code\": \"SESSION_IN_USE\", \"message\": \"Ya hay una
partida en curso en otro dispositivo.\"}"
}

START RequestId: ee75bb39 ...
Acceso concedido a a. Puerta bloqueada.
END RequestId: ee75bb39 ... Duration: 74.98 ms

```

Escenario 3 — Múltiples usuarios en paralelo

Los últimos registros muestran accesos del usuario “abc” intercalados con los del usuario “d”, confirmando que el sistema gestiona sesiones independientes por cuenta sin interferencias entre usuarios distintos.

Listado 6.7: *CloudWatch* — sesiones paralelas de usuarios distintos

```

START RequestId: affde188 ...
Acceso concedido a abc. Puerta bloqueada.
END RequestId: affde188 ... Duration: 46.23 ms

START RequestId: 98acc28a ...

```

```
Acceso concedido a d. Puerta bloqueada.
END RequestId: 98acc28a ... Duration: 93.49 ms
```

Anexo C: Función *Lambda* VerifyGameHash

Trazas de ejecución (*CloudWatch*)

Los *logs* recogen tres escenarios distintos: acceso con un *hash* no existente, acceso con un ejecutable desactualizado y un acceso con la última versión del ejecutable.

Escenario 1 — *Hash* inexistente

Todos los *requests* de la sesión de prueba devuelven INFO undefined en el *log*, lo que corresponde al `console.log(result.Item)` del código cuando el *hash* enviado por el cliente no existe en la tabla `HistoricExeHashes`, `result.Item` es undefined y la función *Lambda* responde con el código DENIED. Esto ocurre porque durante las pruebas el ejecutable se lanza desde el editor de *Unity*, cuyo *hash* no está registrado como versión publicada válida.

Listado 6.8: *CloudWatch* y *Lambda* — *Hash* no registrado

```
START RequestId: c028133e ... Init Duration: 425.04 ms
INFO undefined
END RequestId: c028133e ... Duration: 888.03 ms

Respuesta devuelta por Lambda:
{
  "statusCode": 403,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Content-Type": "application/json"
  },
  "body": "{\"code\":\"DENIED\",\"message\":\"Esta version no esta autorizada.\"}"
}
```

Escenario 2 — *Hash* desactualizado

El *log* muestra tanto el *hash* como el que este no es actual. Esto ocurre ya que se ha ejecutado una versión antigua del programa y por lo tanto el *hash*, que ya no corresponde con la última versión, no se ha borrado pero se ha desmarcado la *flag* `isCurrent`. En este escenario, *Lambda* responde con el código UPDATE_REQUIRED

Listado 6.9: *CloudWatch* y *Lambda* — *Hash* desactualizado

```
START RequestId: 2b47d6b7 ...
isCurrent: { BOOL: false }, clientHash: { S: 'c2f3d552 ...' }
END RequestId: 2b47d6b7 ... Duration: 803.63 ms

Respuesta devuelta por Lambda:
{
  "statusCode": 426,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Content-Type": "application/json"
  },
  "body": "{\"code\":\"UPDATE_REQUIRED\",\"message\":\"Esta version no es la ultima publicada.\"}"
}
```

```

},
"body": "{\"code\":\"UPDATE_REQUIRED\",\"message\":\"Tu version no es la
mas reciente, actualiza el juego.\"}"
}

```

Escenario 3 — Acceso correcto

En el listado de este último posible escenario se observa lo mismo que en el caso anterior a excepción de que la *flag* `isCurrent` es válida, indicando así un registro válido desde un ejecutable actualizado y sin modificar.

Listado 6.10: *CloudWatch* y *Lambda* — Acceso válido

```

START RequestId: 7e59a00b ...
isCurrent: { BOOL: true }, clientHash: { S: '62fbbd30 ...' }
END RequestId: 7e59a00b ... Duration: 65.66 ms

Respuesta devuelta por Lambda:
{
  "statusCode": 200,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Content-Type": "application/json"
  },
  "body": "{\"code\":\"OK\",\"message\":\"Version actual.\"}"
}

```

Tabla `HistoricExeHashes`

A continuación se muestra un ejemplo del contenido de la tabla `HistoricExeHashes`. La función `VerifyGameHash` comprueba los datos de esta y con ello hace las comprobaciones para devolver el código pertinente.

Por orden muestra la lista histórica de *hashes* y si actualmente es válido o no.

clientHash	isCurrent
62fbbd309...	true
c2f3d552a...	false
0948dc407...	false

Cuadro 6.3: Ejemplo de estado de tabla de *DynamoDB* `HistoricExeHashes`

Anexo D: Función *Lambda* `CheckDemoStatus`

Este apartado muestra los casos posibles de acceso al juego con una versión *demo*. Las tres respuestas posibles son: una sesión que supera la fecha límite, un acceso con una cuenta con más tiempo jugado del permitido o una sesión de juego que cumpla con las condiciones

Escenario 1 — Fecha de validez de demo expirada

El *request* mostrado indica que el intento de jugar se ha realizado en una fecha posterior a la delimitada para ese usuario. Los números que aparecen son *epochs*, el

primero corresponde a la fecha actual y el segundo a la límite. En este escenario *Lambda* devuelve el código EXPIRED.

Listado 6.11: *CloudWatch* y *Lambda* — Fecha límite alcanzada

```
START RequestId: 9b8518d8 ...
Date expired: 1777469484 > 1776555512
END RequestId: 9b8518d8 ... Duration: 48.77 ms

Respuesta devuelta por Lambda:
{
  "statusCode": 403,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Content-Type": "application/json"
  },
  "body": "{\"code\":\"EXPIRED\",\"message\":\"Demo caducada.\"}"
}
```

Escenario 2 — Tiempo permitido superado

En este caso, la cuenta que intenta acceder ha consumido el tiempo de dos horas que se le ha otorgado. Los números que aparecen son los segundos consumidos y el límite de segundos. En este escenario *Lambda* devuelve el código TIME_LIMIT_REACHED.

Listado 6.12: *CloudWatch* y *Lambda* — Tiempo de *demo* superado

```
START RequestId: f4e317c2 ...
Time limit reached: 7205 > 7200
END RequestId: f4e317c2 ... Duration: 157.24 ms

Respuesta devuelta por Lambda:
{
  "statusCode": 403,
  "headers": {
    "Access-Control-Allow-Origin": "*",
    "Content-Type": "application/json"
  },
  "body": "{\"code\":\"TIME_LIMIT_REACHED\",\"message\":\"Tiempo de demo agotado.\"}"
}
```

Escenario 3 — Sesión válida

Cuando el jugador accede a la *demo* dentro de los parámetros de fecha y tiempo jugado en los *logs* de *CloudWatch* se muestran el comienzo y final de cada petición, sin información adicional. Al estar la sesión validada, *Lambda* devuelve un código 200.

Listado 6.13: *CloudWatch* y *Lambda* — Sesión válida

```
START RequestId: 85c2cdfb ...
END RequestId: 85c2cdfb ... Duration: 191.86 ms

Respuesta devuelta por Lambda:
{
  "statusCode": 200,
  "headers": {
    "Access-Control-Allow-Origin": "*",
```

```

    "Content-Type": "application/json"
  },
  "body": "{ \"code\": \"OK\", \"message\": \"Acceso permitido.\", \"allowed\": true, \"usedSeconds\": 0, \"limitSeconds\": 7200 }"
}

```

Tabla DemoAccess

La siguiente tabla es un estado de los datos que puede contener DemoAccess, tabla de la cual la función *Lambda* CheckDemoStatus consulta y actualiza los datos.

Por orden muestra el *ID* del usuario, la fecha de caducidad de la *demo*, la primera vez y la última que se ha iniciado sesión (en *epoch*), el tiempo límite y el tiempo empleado.

userId	expiration	firstLogin	lastCheck	playTime	usedTime
302c29fc...	1777450207	1777453515	1777465999	7200	332
20cc19dc...	1777539496	1777453096	1777453127	7200	31
605c496c...	1777476445	1777390045	1777463377	7200	3949
602ca93c...	1777476933	1777390533	1777475205	7200	2291
a01c793c...	1777463067	1777406667	1777474763	7200	1010

Cuadro 6.4: Ejemplo de estado de tabla de *DynamoDB* DemoAccess

Link de página web

En el siguiente [enlace](#) se te pide crear una cuenta con la que podrás descargar el juego. Una vez se abra el ejecutable de *Unity* este se puede probar con la misma cuenta.

Bibliografía

- [1] Newzoo, “Global games market report 2022,” 2022, [Online]. Available: <https://newzoo.com/resources/trend-reports/newzoo-global-games-market-report-2022-free-version>.
- [2] J. Prazak, M. Macas, and T. Cerny, “New usage of telemetry for anti-cheating in fps game,” 2024, [Online]. Available: https://www.researchgate.net/publication/377645285_New_usage_of_telemetry_for_anti-cheating_in_FPS_game.
- [3] OWASP Foundation, “Owasp api security top 10,” 2023, [Online]. Available: <https://owasp.org/API-Security/>.
- [4] Amazon Web Services, *Amazon Cognito Developer Guide*, 2023, [Online]. Available: <https://docs.aws.amazon.com/cognito/>.
- [5] Denuvo Software Solutions, “Denuvo anti-tamper,” 2025, [Online]. Available: <https://irdeto.com/denuvo/>.
- [6] I. Bogost, “The drm debate in pc gaming: Denuvo’s performance and player perception,” *Game Studies Journal*, vol. 18, no. 2, 2018, [Online]. Available: <http://gamestudies.org/1802>.
- [7] BattlEye Innovations, “Battleye anti-cheat,” 2025, [Online]. Available: <https://www.battleye.com/>.
- [8] Amazon Web Services, *Security Overview of AWS Lambda*, 2023, [Online]. Available: <https://docs.aws.amazon.com/lambda/latest/dg/security.html>.
- [9] —, *Controlling and Managing Access to REST APIs in API Gateway*, 2023, [Online]. Available: <https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-control-access-to-api.html>.
- [10] —, *Security Best Practices for Amazon DynamoDB*, 2023, [Online]. Available: <https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/security-best-practices.html>.
- [11] Center for Internet Security, “Cis amazon web services foundations benchmark, v1.4.0,” 2021, [Online]. Available: <https://www.cisecurity.org/benchmark/amazon-web-services>.
- [12] S. Rose, O. Borchert, S. Mitchell, and S. Connelly, “Zero trust architecture,” National Institute of Standards and Technology (NIST), Special Publication 800-207, 2020, [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/specialpublications/NIST.SP.800-207.pdf>.
- [13] OWASP Foundation, “Owasp top 10:2021 – the ten most critical web application security risks,” 2021, [Online]. Available: <https://owasp.org/Top10/>.

- [14] J. H. Saltzer, D. P. Reed, and D. D. Clark, "End-to-end arguments in system design," *ACM Transactions on Computer Systems*, vol. 2, no. 4, pp. 277–288, 1984. [Online]. Available: <https://web.mit.edu/saltzer/www/publications/endtoend/endtoend.pdf>
- [15] Amazon Web Services, "How API Gateway resource policies affect authorization workflow," 2024. [Online]. Available: <https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-authorization-flow.html>
- [16] H. Siamak and T. Mortaza, "Owasp secure by design framework," <https://owasp.org/www-project-secure-by-design-framework/>, [Accessed 04-12-2025]. [Online]. Available: <https://owasp.org/www-project-secure-by-design-framework/>
- [17] F. Nightingale, "Unity real-time development platform vulnerability let attackers execute arbitrary code." [Online]. Available: <https://cybersecuritynews.com/unity-real-time-development-platform-vulnerability/>
- [18] M. Jackson, D. Holladay, and J. Jantunen, "Apex legends migrates to Amazon GameLift servers in just 10 days," May 2025, [Accessed 26-04-2026]. [Online]. Available: <https://aws.amazon.com/blogs/gametech/apex-legends-migrates-to-amazon-gamelift-servers-in-just-10-days/>